



UNIVERSITÄT PADERBORN

Die Universität der Informationsgesellschaft

Fakultät für Elektrotechnik, Informatik und Mathematik

Gruppenbildung für die Aufgabenverteilung in Multiagentensystemen

Bachelorarbeit

im Rahmen des Studiengangs Informatik

zur Erlangung des Grades

Bachelor of Science in Computer Science (B.Sc.-CS)

von

STEPHAN EPP

vorgelegt bei

PROF. DR. HANS KLEINE BÜNING

und

PROF. DR. MATH. FRIEDHELM MEYER AUF DER HEIDE

Paderborn, 30. November 2010

Erklärung

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen worden ist. Alle Ausführungen, die wörtlich oder sinngemäß übernommen worden sind, sind als solche gekennzeichnet.

Ort, Datum

Unterschrift

Inhaltsverzeichnis

1. Einleitung	1
1.1. Problemstellung	1
1.2. Verwandte Arbeiten	1
1.3. Aufgabenbeschreibung	3
1.4. Struktur der Arbeit	4
2. Grundlagen	5
2.1. Agent	5
2.1.1. Charakteristika	5
2.1.2. Umgebungen	6
2.2. Multiagentensystem	7
2.3. Grundlegende Modellierung	7
3. Algorithmus von Shehory und Kraus	9
3.1. Modellierung	9
3.2. Ziel	10
3.3. Set Cover und Set Partition	10
3.4. Disjunkte Gruppen	11
3.4.1. Algorithmus	11
3.4.2. Laufzeitanalyse	13
3.4.3. Approximationsgüte	14
3.5. Überlappende Gruppen	15
3.5.1. Algorithmus	16
3.5.2. Laufzeitanalyse	18
3.5.3. Approximationsgüte	18
3.6. Gegenüberstellung	19
4. Algorithmus von de Weerd et al.	20
4.1. Modellierung	20
4.2. Ziel	21
4.3. Komplexität	21
4.4. Algorithmus	22
4.5. Laufzeitanalyse	23

4.6. Qualität der Lösung	24
5. Implementierung und Simulation	25
5.1. Implementierung	25
5.2. Simulation	25
5.2.1. Algorithmus von Shehory und Kraus	25
5.2.2. Algorithmus von de Weerd et al.	27
6. Zusammenfassung	29
6.1. Fazit	29
6.2. Ausblick	30
A. Anhang	31
A.1. Messergebnisse	31
A.2. Zufalls-Graph	31
A.3. CD	32
Literaturverzeichnis	33

Abbildungsverzeichnis

4.1. Beispiel für einen Graphen $G = (\mathcal{A}, E)$ mit $\mathcal{A} = \{a_1, a_2, \dots, a_7\}$	20
5.1. Laufzeit und Gesamterfolg in Abhängigkeit von k	26
5.2. Laufzeit und Gesamterfolg in Abhängigkeit von p	28
5.3. Veranschaulichung des Graphen G	28
A.1. Laufzeit in Abhängigkeit von n für $k \in \{2, 3, 4\}$	31

1. Einleitung

Multiagentensysteme sind ein wichtiger Forschungsbereich der Künstlichen Intelligenz, der immer mehr an Interesse gewinnt. In [Syc98] erhofft man sich durch Multiagentensysteme neue Entwurfsmuster für die Konzeptionierung, den Entwurf und der Implementierung von Software-Systemen. Insbesondere für das Entwerfen von Systemen in einer verteilten Umgebung, wie zum Beispiel das Internet, bilden Multiagentensysteme ein geeignetes Modell. Auch reale Szenarien lassen sich durch Multiagentensysteme simulieren. Hierzu ein kleines Beispiel.

Unmittelbar nach einer Naturkatastrophe wollen Helfer hilfebedürftigen Menschen vor Ort helfen. Die Helfer haben jeweils begrenzte Ressourcen, wie zum Beispiel Fachwissen und Medikamenten-Vorrat. Es gibt viele Aufgaben, die die Helfer zu bearbeiten haben, um den hilfebedürftigen Menschen helfen zu können. Die Bearbeitung einer Aufgabe erfordert Ressourcen, die durchaus größer sein können als die Ressourcen eines Helfers. Die Helfer haben die Möglichkeit, Gruppen zu bilden bzw. zusammen zu arbeiten, so dass sie dann gemeinsam eine Aufgabe bearbeiten können. Dabei sollten die Helfer so zusammenarbeiten, dass die Aufgaben möglichst effizient bearbeitet werden.

1.1. Problemstellung

Gegeben sei eine Menge von Agenten und eine Menge von zu bearbeitenden Aufgaben in einem Multiagentensystem. Die Agenten haben beschränkte Ressourcen. Die Bearbeitung einer Aufgabe erfordert Ressourcen. Kann ein Agent auf Grund seiner beschränkten Ressourcen eine Aufgabe nicht bearbeiten, kann er durch die Ressourcen der anderen Agenten unterstützt werden, so dass die Aufgabe bearbeitet werden kann. Durch Gruppenbildung bzw. durch das Zusammenarbeiten unterstützen die Agenten sich gegenseitig bei der Bearbeitung einer Aufgabe. Die Gruppenbildung bzw. das Zusammenarbeiten soll dabei so durchgeführt werden, dass die Ressourcen einer Gruppe möglichst optimal zu den Ressourcen der Aufgabe passen, so dass möglichst wenig Ressourcen verschwendet werden und die Aufgabe effizient bearbeitet wird.

1.2. Verwandte Arbeiten

Gruppenbildung für die Aufgabenverteilung (Coalition Formation for Task Allocation, CFTA) in Multiagentensystemen wurde u.a. von Shehory und Kraus in [SK93] bear-

beitet. Sie gehen in [SK93] davon aus, dass die Agenten nur dann zusammenarbeiten, wenn der eigene Erfolg eines Agenten bei Zusammenarbeit erhöht wird. Gesucht sind jedoch Agenten, die den Gesamterfolg des Multiagentensystems, in dem sich die Agenten befinden, maximieren.

In [SK95] wird gezeigt, wie stark das CFTA Problem zusammenhängt mit dem Set Cover Problem (SCP) und dem Set Partition Problem (SPP). Kozen zeigt in [Koz92], dass das SPP *NP*-hart ist. Karp klassifiziert in [Kar72] das SCP und das SPP als *NP*-vollständig, womit das optimale Lösen dieser Probleme zu exponentieller Laufzeit führt und damit praktisch für große Eingaben nicht möglich ist, sofern $NP \neq P$. Shehory und Kraus stellen in [SK95] einen verteilten Algorithmus vor, der in polynomieller Laufzeit die Ressourcenauslastung der Agenten approximiert. Mit diesem Algorithmus bilden die Agenten selbstständig - also ohne die Hilfe einer zentralen Instanz - Gruppen, denen eine möglichst passende Aufgabe zugewiesen wird.

Der Algorithmus besteht aus zwei wesentlichen Schritten. Im ersten Schritt bestimmt jeder Agent zunächst alle potentiellen Gruppen. Da eine Gruppe nicht mehr als aus k Agenten besteht, ist die Menge aller potentiellen Gruppen (Menge aller möglichen k -beschränkten Teilmengen von der Menge der Agenten) eines Agenten nicht exponentiell groß in der Anzahl der Agenten. Für diese Gruppen wird anschließend jeweils ein Gruppen-Wert ermittelt. Dabei wird der Gruppen-Wert so berechnet, dass gilt: Je kleiner die nicht benötigten Ressourcen der Gruppe bei Bearbeitung einer bestimmten Aufgabe sind bzw. je besser die Ressourcenauslastung der Gruppe ist, desto größer wird der Gruppen-Wert. Im zweiten Schritt werden die Gruppen mit Hilfe der berechneten Gruppen-Werte gebildet. Dazu bestimmt jeder Agent für sich den größten Gruppen-Wert und gibt diesen allen anderen Agenten bekannt. Unter allen bekannt gegebenen Gruppen-Werten wird dann von allen Agenten der größte Gruppen-Wert V gewählt. Zu jedem Gruppen-Wert korrespondieren eine Gruppe und eine zu bearbeitende Aufgabe. Die Gruppe C mit dem größten Gruppen-Wert wird von den Agenten gebildet und ihr die korrespondierende Aufgabe zugewiesen. Danach wird dieser Gruppe C das Gewicht $w = \frac{1}{V|C|}$ zugeordnet. Gruppen werden gebildet, solange noch Aufgaben verteilt werden können. Der Gesamterfolg des Multiagentensystems besteht in der Summe der Gruppen-Werte der gebildeten Gruppen. Je größer der Gesamterfolg ist, desto besser ist die Ressourcenauslastung.

Service und Adams weisen in [SA10] jeder Aufgabe zusätzlich einen Nutzen-Wert zu. Sie bemerken, dass es keinen Algorithmus gibt, der in polynomieller Laufzeit den Gesamt-Nutzen besser als linear in der Anzahl der Aufgaben approximieren kann. Dabei ergibt sich der Gesamt-Nutzen aus der Summe der Nutzen-Werte der bearbeiteten Aufgaben. Shehory und Kraus zeigen jedoch mit ihrem Algorithmus eine logarithmische Approximationsgüte in der Anzahl der Agenten bzgl. der Ressourcenauslastung.

In [SK98] gehen Shehory und Kraus auf zwei verschiedene Ansätze zur Bildung von

Gruppen ein. Im ersten Ansatz dürfen Agenten nur in einer Gruppe enthalten sein. Es werden also disjunkte Gruppen gefordert. Das hat den Vorteil, dass ein Agent die Berechnung zur Bildung der Gruppen beenden kann, sobald er selber einer Gruppe zugeordnet worden ist. Im zweiten Ansatz dürfen Agenten in mehreren Gruppen enthalten sein. Hier werden also überlappende Gruppen zugelassen. Das hat den Vorteil, dass Agenten ihre Ressourcen zur Bearbeitung der Aufgaben auf mehrere Gruppen aufteilen können. Die nicht benötigten Ressourcen der Gruppen werden dadurch verringert und der jeweilige Gruppen-Wert sowie der Gesamterfolg des Multiagentensystems erhöht.

In [SK96] wird darauf hingewiesen, dass im Fall der überlappenden Gruppen das Gewicht einer Gruppe C berechnet werden muss mit $w = \frac{1}{V_z}$. Dabei ist V der Gruppen-Wert und z die Anzahl der neuen Agenten in C . Neue Agenten sind die Agenten, die zum ersten mal einer Gruppe beitreten.

Shehory und Kraus sowie Service und Adams nehmen an, dass ein Agent mit jedem anderen Agenten kommunizieren kann. In [TA05] und [dZK07] werden Agenten durch Knoten in einem ungerichteten Graphen repräsentiert. Genau dann, wenn zwei Knoten in diesem Graphen durch eine Kante verbunden sind, können die repräsentierten Agenten miteinander kommunizieren. de Weerd, Zhang und Klos nehmen in [dZK07] an, dass ein Agent bei der Bearbeitung einer Aufgabe nur durch seine Nachbarn unterstützt werden kann. Weiterhin gehen sie davon aus, dass die Menge der Aufgaben auf alle Agenten verteilt wird. Jeder Agent kennt also nur die Aufgaben, die auch in seiner Nachbarschaft verteilt worden sind. Außerdem wird jeder Aufgabe ein Nutzen-Wert zugewiesen. Sie stellen in [dZK07] einen verteilten Algorithmus vor, mit dem die Agenten zusammen Aufgaben bearbeiten, die nur lokal bekannt sind.

Mit diesem Algorithmus bestimmen die Agenten zunächst die effizienteste Aufgabe in ihrer Nachbarschaft. Die Effizienz einer Aufgabe berechnet sich aus dem Quotienten des Nutzen-Wertes und der erforderlichen Ressourcen der Aufgabe. Darauf hin macht jeder Agent dem Agenten aus seiner Nachbarschaft, dem bei der Aufgabenverteilung die effizienteste Aufgabe zugeteilt worden ist, ein Angebot. Das Angebot beinhaltet alle Ressourcen, die ein Agent für diese Aufgabe zur Bearbeitung zur Verfügung stellen kann. Wenn ausreichend Ressourcen zur Bearbeitung der Aufgabe vorhanden sind, wird die Aufgabe bearbeitet und anschließend gelöscht. Wenn keine weiteren Ressourcen aus der Nachbarschaft zur Bearbeitung dieser Aufgabe verfügbar sind, wird die Aufgabe gelöscht. Solange noch Aufgaben in der Nachbarschaft vorhanden sind, versuchen die Agenten die Aufgaben zu bearbeiten.

1.3. Aufgabenbeschreibung

Die Aufgabe dieser Arbeit besteht darin, zunächst einen Überblick über das Coalition Formation for Task Allocation (CFTA) Problem zu geben. Mit dem Set Cover Problem

und dem Set Partition Problem soll verdeutlicht werden, dass das optimale Lösen des CFTA Problems schwer ist. In [SK98] geben Shehory und Kraus zwei Ansätze an, mit denen die Agenten selbstständig Gruppen bilden, denen dann auch jeweils eine Aufgabe zugewiesen wird. Für beide Ansätze soll das Modell von Shehory und Kraus beschrieben und implementiert werden, so dass diese Ansätze zur Lösung des CFTA Problems umgesetzt werden können. Dazu soll die Laufzeit und die Approximationsgüte beider Ansätze analysiert werden.

Im ersten Ansatz werden nur disjunkte Gruppen zugelassen. Ein Agent darf also nur einer Gruppe beitreten. Im zweiten Ansatz werden überlappende Gruppen zugelassen, so dass ein Agent mehreren Gruppen beitreten darf. In [SK98] bemerken Shehory und Kraus, dass eine Funktion bzw. Methode notwendig ist, mit Hilfe derer die Agenten ihre Ressourcen auf verschiedene Gruppen aufteilen können, um dadurch mehreren Gruppen beitreten zu können. Das Problem zur Aufteilung der Ressourcen eines Agenten auf mehrere Gruppen soll gelöst und umgesetzt werden, so dass beide Ansätze gegenübergestellt und bzgl. ihrer Laufzeit und Approximationsgüte bewertet werden können.

In [TA05] und [dZK07] werden Agenten durch Knoten in einem ungerichteten Graphen repräsentiert. Hier soll untersucht werden, wie effizient die Aufgaben durch die Agenten bearbeitet werden können, wenn dabei jeder Agent nur einen Teil der anderen Agenten und Aufgaben kennt. Dazu soll der Algorithmus von de Weerd et al. aus [dZK07] beschrieben, bzgl. der Laufzeit analysiert und implementiert werden.

1.4. Struktur der Arbeit

Zunächst werden in Kapitel 2 die Begriffe Agent und Multiagentensystem definiert und anschließend eine grundlegende Modellierung eingeführt. In Kapitel 3 wird der Algorithmus von Shehory und Kraus vorgestellt und in Kapitel 4 der Algorithmus von de Weerd et al.. In Kapitel 5 werden die Implementierung der Algorithmen und die Ergebnisse der Simulation der implementierten Algorithmen vorgestellt. Abschließend wird in Kapitel 6 die Arbeit zusammengefasst, ein Fazit gezogen und ein Ausblick für weitere Arbeiten gegeben.

2. Grundlagen

In diesem Kapitel werden zunächst die Begriffe Agent und Multiagentensystem definiert. Anschließend wird eine grundlegende Modellierung für den Algorithmus von Shehory und Kraus und für den Algorithmus von de Weerd et al. eingeführt.

2.1. Agent

Russel und Norvig betrachten in [RN04] einen Agenten als alles, was seine Umgebung wahrnehmen und in dieser Umgebung entsprechend handeln kann. Ein Agent kann zum Beispiel als menschlicher Agent oder als Roboter-Agent oder auch als Software-Agent verstanden werden. Ein menschlicher Agent hat Augen und Ohren um seine Umwelt wahrzunehmen und kann mit seinen Füßen und Händen entsprechend handeln. Ein Roboter-Agent kann zum Beispiel über Kameras und Infrarotbereichsmelder seine Umwelt wahrnehmen und durch den Antrieb von Motoren entsprechend handeln. Ein Software-Agent kann Dateien oder Netzwerkpakete entgegen nehmen bzw. wahrnehmen und entsprechende Bildschirmausgaben erzeugen oder Netzwerkpakete versenden.

Wooldridge merkt an, dass es keine allgemein gültige Definition für einen Agenten gibt und dass die Definition eines Agenten abhängig ist von dem Kontext bzw. der Umgebung, in der ein Agent eingesetzt wird. Wooldridge versucht dennoch, in [Woo09, Seite 21] eine gebrauchstaugliche Definition für einen Agenten zu geben.

An agent is a computer system that is situated in some environment, and that is capable of autonomous action in this environment in order to meet its delegated objectives.

Ein **Agent** führt also in seiner Umgebung selbstständig Aktionen aus. Dabei werden die Aktionen mit dem Ziel ausgeführt, für das der Agent bestimmt ist.

2.1.1. Charakteristika

Ein Agent kann durch verschiedene Charakteristika beschrieben werden. Wooldridge beschreibt in [Woo98] einen Agenten durch folgende Charakteristika.

Autonomie: Ein autonomer Agent arbeitet ohne den direkten Eingriff eines Menschen und hat die vollständige Kontrolle über seine Aktionen.

Soziale Fähigkeiten: Ein Agent mit sozialen Fähigkeiten ist zum Beispiel dazu fähig, mit Menschen oder anderen Agenten zu kooperieren, um das für ihn bestimmte Ziel zu erreichen.

Reaktivität bzw. Proaktivität: Ein reaktiver Agent erkennt Veränderungen in seiner Umgebung und reagiert durch entsprechende Aktionen. Ein proaktiver Agent kann aus interner Motivation heraus selbstständig aktiv werden.

Wahrhaftigkeit: Ein wahrhaftiger Agent wird nicht bewusst falsche Informationen an andere Agenten oder Menschen weitergeben.

Selbst- bzw. Gruppen-Rationalität: Ein selbst-rationaler Agent verfolgt nur das eigene Ziel. Ein gruppen-rationaler Agent verfolgt das gemeinsame Ziel von einer Menge von Agenten.

Lernfähigkeit: Ein lernfähiger Agent ist fähig, sich an seine Umwelt anzupassen.

Wooldridge hat bereits in der Definition eines Agenten die Charakteristik Autonomie erwähnt. Autonomie ist somit eine der wichtigsten Eigenschaften eines Agenten. Wooldridge macht aber auch darauf aufmerksam, dass die Eigenschaften eines Agenten von der Umgebung abhängig sind, in der ein Agent eingesetzt wird.

2.1.2. Umgebungen

Es gibt verschiedene Arten von Umgebungen, in denen sich ein Agent befindet kann. Wooldridge klassifiziert in [Woo09] die möglichen Umgebungen wie folgt.

Erfassbar bzw. nicht erfassbar: Eine erfassbare Umgebung ist eine Umgebung, in der sich ein Agent über den Zustand der Umgebung informieren kann. Für den Agenten sind also vollständige, exakte und aktuelle Informationen der Umgebung erfassbar. Er kann sich zum Beispiel darüber informieren, wie viele Agenten mit ihm und wie viele zu bearbeitende Aufgaben sich in der Umgebung befinden.

Deterministisch bzw. nichtdeterministisch: In einer deterministischen Umgebung löst jede Aktion eines Agenten genau einen Effekt aus, der sich auf den Zustand der Umgebung auswirkt. Es ist also klar, in welchen Zustand die Umgebung nach einer bestimmten Aktion eines Agenten übergehen wird.

Statisch bzw. dynamisch: Eine statische Umgebung ist eine Umgebung, deren Zustand sich nur in Abhängigkeit der Aktion, die ein Agent in ihr ausführt, ändert. Eine dynamische Umgebung ist eine Umgebung, in der auch andere Ereignisse bzw. Prozesse den Zustand der Umgebung ändern können. Zum Beispiel kann sich die Temperatur in der Umgebung erhöhen, wobei die Erhöhung der Temperatur nicht durch die Aktion eines Agenten ausgelöst wurde.

Diskret bzw. kontinuierlich: Eine Umgebung ist diskret, wenn für diese Umgebung eine feste, endliche Anzahl an ausführbaren Aktionen existiert. Eine Umgebung ist kontinuierlich, wenn für diese Umgebung unendlich viele ausführbare Aktionen existieren.

Wooldridge merkt an, dass die komplexeste Art von Umgebung diejenige ist, die nicht erfassbar, nichtdeterministisch, dynamisch und kontinuierlich ist. Shehory und Kraus gehen davon aus, dass die Agenten sich in einer erfassbaren Umgebung befinden. Jeder Agent kennt alle anderen Agenten und Aufgaben. In [Syc98] charakterisiert Sycara eine Umgebung jedoch unter anderem damit, dass jeder Agent nur unvollständige Informationen hat, mit Hilfe derer er ein bestimmtes Problem lösen bzw. ein bestimmtes Ziel erreichen kann. Agenten in dieser Umgebung befinden sich also in einer nicht erfassbaren Umgebung. de Weerd et al. gehen davon aus, dass die Agenten sich in einer nicht erfassbaren Umgebung befinden. Die Agenten kennen nur einen Teil der Agenten und Aufgaben in ihrer Umgebung.

2.2. Multiagentensystem

Mit der vorangegangenen Definition des Begriffes Agent wird nun der Begriff Multiagentensystem definiert. Wooldridge definiert in [Woo09, Seite 5] ein Multiagentensystem wie folgt.

A multiagent system is one that consists of a number of agents, which interact with one another, typically by exchanging messages through some computer network infrastructure.

Ein **Multiagentensystem** besteht also aus einer Anzahl von Agenten, die miteinander interagieren, indem sie Nachrichten austauschen. Somit stellt ein Multiagentensystem die Umgebung dar, in der sich die Agenten befinden. Je nachdem, um welche Klasse von Umgebung bzw. Multiagentensystem es sich handelt und für welche Ziele die Agenten bestimmt sind, interagieren die Agenten durch den Austausch von Nachrichten und führen entsprechende Aktionen aus. Sowohl mit dem Algorithmus von Shehory und Kraus als auch mit dem Algorithmus von de Weerd et al. tauschen die Agenten Nachrichten aus. Das Ziel, für das die Agenten in dem Modell von Shehory und Kraus und in dem Modell von de Weerd et al. bestimmt sind, ist in Abschnitt 3.2 und Abschnitt 4.2 formuliert.

2.3. Grundlegende Modellierung

Mit den vorangegangenen Definitionen der Begriffe Agent und Multiagentensystem soll nun eine Modellierung eingeführt werden, die die Grundlage für das Modell von Shehory und Kraus und für das Modell von de Weerd et al. legt.

Gegeben sei eine Umgebung bzw. ein Multiagentensystem, in dem sich eine Menge von Agenten und eine Menge von Aufgaben befinden. Die Menge der **Agenten** $\mathcal{A}$ sei gegeben durch $\mathcal{A} = \{a_1, \dots, a_n\}$. Ein Agent $a_i \in \mathcal{A}$ hat die **Ressourcen** $B_i = (b_1^i, \dots, b_r^i)$ mit $b_p^i \in \mathbb{R}_{\geq 0}$ für $p = 1, \dots, r$. Setzt ein Agent einen Teil seiner Ressourcen zur Bearbeitung einer Aufgabe ein, kann er diesen Teil seiner Ressourcen nicht wieder verwenden. Die Agenten seien charakterisiert durch Autonomie, Wahrhaftigkeit und Gruppen-Rationalität. Die Agenten werden also selbstständig arbeiten, keine falschen Informationen weitergeben und das gemeinsame Ziel aller Agenten verfolgen. Tauschen die Agenten Nachrichten aus, wird angenommen, dass keine Zeit für die Nachrichtenübertragung benötigt wird. Die gesendeten Nachrichten kommen also sofort beim Empfänger an. Die Menge der **Aufgaben** $\mathcal{T}$ sei gegeben durch $\mathcal{T} = \{t_1, \dots, t_m\}$. Die Bearbeitung einer Aufgabe $t_j \in \mathcal{T}$ erfordert die Ressourcen $B_j = (b_1^j, \dots, b_r^j)$ mit $b_p^j \in \mathbb{R}_{\geq 0}$. Das Multiagentensystem sei diskret, so dass die Agenten nur die Aktionen im Multiagentensystem ausführen, die auch in den Algorithmen von Shehory und Kraus sowie von de Weerd et al. gefordert werden.

3. Algorithmus von Shehory und Kraus

In diesem Kapitel wird zunächst die Modellierung aus Abschnitt 2.3 ergänzt, so dass das Ziel des Algorithmus von Shehory und Kraus definiert werden kann. Der Zusammenhang zwischen dem Ziel und dem Set Cover Problem und dem Set Partition Problem wird verdeutlicht und anschließend der Algorithmus vorgestellt.

3.1. Modellierung

Die Modellierung aus Abschnitt 2.3 wird so ergänzt, dass das Multiagentensystem, in dem sich die Agenten $\mathcal{A}$ und Aufgaben $\mathcal{T}$ befinden, als ein erfassbares System aufgefasst wird. Es wird also angenommen, dass jeder Agent alle anderen Agenten und alle Aufgaben kennt. Jeder Agent hat also Zugriff auf alle Aufgaben und kann mit jedem anderen Agenten kommunizieren.

Ein Agent $a_i \in \mathcal{A}$ hat die Ressourcen $B_i = (b_1^i, \dots, b_r^i)$ mit $b_p^i \in \mathbb{R}_{\geq 0}$ für $p = 1, \dots, r$. Die Bearbeitung einer Aufgabe $t_j \in \mathcal{T}$ erfordert die Ressourcen $B_j = (b_1^j, \dots, b_r^j)$ mit $b_p^j \in \mathbb{R}_{\geq 0}$. Da die Bearbeitung einer Aufgabe die Ressourcen eines Agenten übersteigen kann, haben die Agenten die Möglichkeit, Gruppen zu bilden, in denen sie jeweils eine Aufgabe bearbeiten können. Die **Gruppen-Ressourcen** B_{C_ν} einer Gruppe $C_\nu \subseteq \mathcal{A}$ sind gegeben durch $B_{C_\nu} = \sum_{a_i \in C_\nu} B_i = (b_1^{C_\nu}, \dots, b_r^{C_\nu})$ (Summe von Vektoren). Eine Aufgabe $t_j \in \mathcal{T}$ ist durch die Gruppe C_ν genau dann **bearbeitbar**, wenn $b_p^j \leq b_p^{C_\nu}$ für $p = 1, \dots, r$. Jede Gruppe C_ν hat einen **Gruppen-Wert**. Der Gruppen-Wert einer Gruppe C_ν wird bezeichnet mit V_ν und ist genau dann maximal für eine Aufgabe $t_j \in \mathcal{T}$, wenn $b_p^j = b_p^{C_\nu}$ für $p = 1, \dots, r$. Je besser also die Ressourcen einer Aufgabe zu den Ressourcen einer Gruppe C_ν passen, desto größer ist der Gruppen-Wert V_ν .

Für die Analyse der Approximationsgüte in Abschnitt 3.4.3 und Abschnitt 3.5.3 wird jeder Gruppe C_ν ein **Kosten-Wert** $c_\nu = \frac{1}{V_\nu}$ zugeordnet. Je besser also die Ressourcenauslastung von C_ν ist, desto geringer sind die Kosten c_ν für C_ν . Eine **Gruppen-Konfiguration** $\mathcal{C}$ enthält alle Gruppen C_ν , die durch die Agenten gebildet werden, also $\mathcal{C} = \{C_1, C_2, \dots\}$ mit $|\mathcal{C}| \leq n$. Der **Gesamterfolg** des Multiagentensystems ist gegeben durch $\sum_{C_\nu \in \mathcal{C}} V_\nu$, wobei V_ν der Gruppen-Wert von C_ν ist. Je größer der Gesamterfolg ist, desto besser ist die Ressourcenauslastung jeder Gruppe $C_\nu \in \mathcal{C}$.

3.2. Ziel

Mit der eingeführten Modellierung lässt sich das Ziel, für das die Agenten bestimmt sind, nun formulieren. Gesucht ist eine Gruppen-Konfiguration $\mathcal{C}$, deren Gruppen C_ν jeweils eine Aufgabe zugewiesen werden, so dass $\sum_{C_\nu \in \mathcal{C}} V_\nu$ maximal (bzw. $\sum_{C_\nu \in \mathcal{C}} c_\nu$ minimal) wird mit $\bigcup_{C_\nu \in \mathcal{C}} C_\nu = \mathcal{A}$. Für disjunkte Gruppen wird zusätzlich gefordert, dass $C_\nu \cap C_\mu = \emptyset$ für alle $C_\nu, C_\mu \in \mathcal{C}$ und $\nu \neq \mu$.

3.3. Set Cover und Set Partition

Shehory und Kraus zeigen in [SK95], wie stark das Coalition Formation for Task Allocation Problem mit dem Set Cover Problem (SCP) und dem Set Partition Problem (SPP) übereinstimmt. Formulieren lassen sich die beiden Probleme wie folgt. Gegeben sei eine Menge $\mathcal{A} = \{a_1, \dots, a_n\}$ und eine Menge $\mathcal{S}$, die alle möglichen Teilmengen von $\mathcal{A}$ enthält. Jede Menge $C_\nu \in \mathcal{S}$ habe einen Kosten-Wert c_ν . Gesucht ist eine Teilmenge $\mathcal{C} \subseteq \mathcal{S}$, die alle Elemente von $\mathcal{A}$ so überdeckt, dass $\sum_{C_\nu \in \mathcal{C}} c_\nu$ minimal wird mit $\bigcup_{C_\nu \in \mathcal{C}} C_\nu = \mathcal{A}$. Für das SPP wird zusätzlich gefordert, dass $C_\nu \cap C_\mu = \emptyset$ für alle $C_\nu, C_\mu \in \mathcal{C}$ und $\nu \neq \mu$ (vgl. mit Abschnitt 3.2).

Karp klassifiziert in [Kar72] das SCP und das SPP (in [Kar72] Exact Cover) als *NP*-vollständig. Für den Nachweis, dass das SCP *NP*-hart ist, gibt Karp in [Kar72] die Idee an, mit der das Node Cover Problem (auch bekannt als Vertex Cover Problem) auf das SCP in polynomieller Zeit reduziert werden kann. Außerdem gibt er in [Kar72] auch den Hinweis, mit dem sich das Chromatic Number Problem (auch bekannt als Graph Coloring Problem) auf das SPP in polynomieller Zeit reduzieren lässt.

Bisher wurde noch kein Algorithmus gefunden, der das SCP oder das SPP in polynomieller Laufzeit optimal lösen kann. Wenn ein Algorithmus existieren würde, der zum Beispiel das SCP in polynomieller Zeit optimal lösen kann, dann würde es auch einen Algorithmus geben, der das Vertex Cover Problem (VCP) in polynomieller Zeit optimal lösen kann. Denn die Aussage der polynomiellen Reduktion (mit der das VCP in polynomieller Zeit auf das SCP reduziert wird) ist, dass das VCP nicht schwerer zu lösen ist als das SCP. Das VCP wäre damit in polynomieller Zeit lösbar. Da das VCP jedoch *NP*-vollständig ist [Kar72], ist auch jedes Problem aus *NP* auf das VCP in polynomieller Zeit reduzierbar. Jedes Problem aus *NP* ist also nicht schwerer zu lösen als das VCP. Somit wäre auch jedes Problem aus *NP* in polynomieller Zeit lösbar bzw. $NP = P$ (da $P \subseteq NP$).

Sofern $NP \neq P$, gibt es jedoch keinen Algorithmus, der das SCP oder das SPP in polynomieller Zeit optimal lösen kann. Dazu wird die Idee betrachtet, in der über jede Menge $C_\nu \in \mathcal{S}$ iteriert wird und C_ν gegebenenfalls zu $\mathcal{C}$ hinzugefügt wird. Da $\mathcal{S}$ jedoch 2^n Mengen enthält, führt diese Idee zu exponentieller Laufzeit in der Anzahl der Elemente von $\mathcal{A}$. Shehory und Kraus führen eine Grenze k ein, die angibt, wie groß jede mögliche

Teilmenge von $\mathcal{A}$ höchstens sein darf. Dadurch gibt es nur noch $\mathcal{O}(n^k)$ mögliche k -beschränkte Teilmengen von $\mathcal{A}$ (vgl. mit Abschnitt 3.4.2). Shehory und Kraus stellen einen Algorithmus vor, mit dem über alle möglichen k -beschränkten Teilmengen von $\mathcal{A}$ iteriert wird.

3.4. Disjunkte Gruppen

Im ersten Ansatz sollen nur disjunkte Gruppen zugelassen werden. Agenten dürfen also nur einer Gruppe beitreten.

3.4.1. Algorithmus

Der Algorithmus von Shehory und Kraus besteht aus zwei wesentlichen Schritten und wird realisiert durch die Algorithmen 3.4.1, 3.4.2 und 3.4.3.

Im ersten Schritt wird von jedem Agenten der Gruppen-Wert einer jeden zuvor bestimmten potentiellen k -beschränkten Gruppe berechnet. Im zweiten Schritt werden die Gruppen gebildet. Bevor die Gruppen-Werte durch jeden Agenten a_i lokal berechnet werden können, muss jeder Agent zunächst bestimmen, für welche potentiellen Gruppen die Gruppen-Werte berechnet werden müssen.

Algorithmus 3.4.1: Bestimmung aller potentiellen Gruppen

```

1 Erstelle eine Liste  $L$ , die alle möglichen  $k$ -beschränkten Gruppen enthält
2 foreach  $a_j \in \mathcal{A} \setminus \{a_i\}$  do
3   Kontaktiere  $a_j$  und speichere die Ressourcen  $B_j$ 
4   foreach Agent  $a_l$ , der  $a_i$  kontaktiert do
5     Lösche  $a_l$  aus  $\mathcal{A}$ 
6     Lösche alle Gruppen aus  $L$ , in denen  $a_l$  enthalten ist
7   end
8 end
9 Berechne für jede Gruppe  $C$  von  $L$  den Gruppen-Wert mit Algorithmus 3.4.2
```

Die Menge aller möglichen k -beschränkten Gruppen wird in Zeile 1 durch die Menge aller möglichen k -beschränkten Teilmengen von $\mathcal{A}$ ermittelt. In Zeile 3 speichert Agent a_i die Ressourcen der Agenten, zu denen er Kontakt aufgenommen hat. Mit den gespeicherten Ressourcen berechnet a_i später den Gruppen-Wert aller Gruppen von L . Für jeden Agenten a_l , der a_i kontaktiert, werden alle Gruppen von L , in denen a_l enthalten ist, in Zeile 6 gelöscht. Für diese Gruppen muss a_i den Gruppen-Wert nicht mehr berechnen, da a_l für diese Gruppen den Gruppen-Wert berechnet. Deshalb wird Agent a_l aus $\mathcal{A}$ in Zeile 5 gelöscht. In Zeile 9 wird dann für jede potentielle Gruppe der Liste L der Gruppen-Wert mit Algorithmus 3.4.2 berechnet.

Algorithmus 3.4.2: Berechnung der Gruppen-Werte

-
- 1 Lösche jede Aufgabe aus $\mathcal{T}$, welche nicht durch Gruppe C bearbeitet werden kann
 - 2 **foreach** $t_j \in \mathcal{T}$ **do**
 - 3 Fasse alle benötigten Ressourcen von t_j als Summe zusammen zu τ_j
 - 4 Fasse alle nicht benötigten Ressourcen von C als Summe zusammen zu δ_j
 - 5 **end**
 - 6 Bestimme für jedes j die maximale Differenz $\tau_j - \delta_j$ als Gruppen-Wert V von C
-

In Zeile 1 wird durch das paarweise Vergleichen der benötigten Ressourcen $B_j = (b_1^j, \dots, b_r^j)$ und der Gruppen-Ressourcen $B_C = (b_1^C, \dots, b_r^C)$ der Gruppe C festgestellt, ob Aufgabe t_j durch C bearbeitet werden kann. In Zeile 3 werden alle benötigten Ressourcen von t_j als Summe zu τ_j zusammengefasst. In Zeile 4 werden alle nicht benötigten Ressourcen von C als Summe zu δ_j zusammengefasst. Für jede bearbeitbare Aufgabe t_j kann mit den Zeilen 2 - 5 ein potentieller Gruppen-Wert als Differenz von $\tau_j - \delta_j$ bestimmt werden. In Zeile 6 wird dann das Maximum aller berechneten Differenzen $\tau_j - \delta_j$ als Gruppen-Wert V von C bestimmt. Denn die größte Differenz $\tau_j - \delta_j$ impliziert die geringsten nicht verwendeten Ressourcen bzw. die beste Ressourcenauslastung von C bei Bearbeitung der Aufgabe t_j .

Nachdem jeder Agent die potentiellen Gruppen bestimmt und ihre Werte berechnet hat, werden die Gruppen mit Algorithmus 3.4.3 gebildet. Dabei geht jeder Agent a_i wie folgt vor.

Algorithmus 3.4.3: Bildung der Gruppen

-
- 1 **while** $\mathcal{T} \neq \emptyset$ und $L \neq \emptyset$ **do**
 - 2 Bestimme den größten Gruppen-Wert und gib diesen allen Agenten bekannt
 - 3 Bestimme den größten Gruppen-Wert V aller Agenten
 - 4 Wähle die zu V korrespondierende Gruppe C und Aufgabe t
 - 5 **if** $a_i \in C$ **then**
 - 6 Gewichte die Gruppe C mit dem Gewicht $w = \frac{1}{\sqrt{z}}$ mit $z = |C|$
 - 7 Trete der Gruppe C mit Aufgabe t bei
 - 8 **else**
 - 9 Lösche alle Gruppen aus L , die mindestens einen Agenten aus C enthalten
 - 10 Aktualisiere mit Algorithmus 3.4.2 die Gruppen-Werte der Gruppen, bei denen t ausgewählt wurde
 - 11 Lösche t aus $\mathcal{T}$
 - 12 **end**
 - 13 **end**
-

Solange noch nicht zugewiesene Aufgaben und potentielle Gruppen existieren, werden in den Zeilen 1 - 13 sukzessive die Gruppen gebildet und den entsprechenden Aufgaben zugewiesen. Dabei gibt jeder Agent a_i in Zeile 2 den größten Gruppen-Wert allen anderen

Agenten bekannt. In Zeile 3 wird der größte Gruppen-Wert V unter allen bekannt gegebenen Gruppen-Werten bestimmt. In Zeile 4 wird die zu V korrespondierende Gruppe C und Aufgabe t gewählt. Die Korrespondenz ist gegeben durch die Zeile 6 von Algorithmus 3.4.2. Mit V geben den Agenten also implizit auch die korrespondierende Gruppe C und Aufgabe t bekannt. Für eine anschauliche Beschreibung der Algorithmen ist jedoch auf eine zusätzliche Indizierung für V verzichtet worden. Falls a_i nun in C enthalten ist, wird C mit $w = \frac{1}{V|C|}$ in Zeile 6 gewichtet und der Gruppe C mit Aufgabe t in Zeile 7 beigetreten. Ansonsten werden in Zeile 9 alle Gruppen gelöscht, in denen mindestens ein Agent aus C enthalten ist. In Zeile 10 werden die Gruppen-Werte aktualisiert, da Aufgabe t nicht mehr zugewiesen werden kann. Damit ist für alle Gruppen, die Aufgabe t als Aufgabe mit den wenigsten nicht benötigten Ressourcen ausgewählt haben, eine Neuberechnung der Gruppen-Werte erforderlich. In Zeile 11 wird die Aufgabe t gelöscht.

3.4.2. Laufzeitanalyse

Die Analyse der Laufzeit ist teilweise angelehnt an [SK95]. Sei dazu $T(m, n)$ die erforderte Laufzeit bei Ausführung der Algorithmen 3.4.1 und 3.4.3.

In Zeile 1 von Algorithmus 3.4.1 werden alle möglichen k -beschränkten Teilmengen von $\mathcal{A}$ berechnet. Es gibt $\binom{n}{s}$ unterschiedliche k -beschränkte Teilmengen von $\mathcal{A}$ für $s = 1, \dots, k$. Sei $K(n)$ die Anzahl aller möglichen k -beschränkten Teilmengen von $\mathcal{A}$. Dann gilt:

$$K(n) = \sum_{s=1}^k \binom{n}{s} = \sum_{s=1}^k \frac{n!}{s!(n-s)!}. \quad (3.4.1)$$

Für $k \leq \frac{n}{2}$ lässt sich $K(n)$ abschätzen durch

$$K(n) < k \cdot \frac{n!}{k!(n-k)!}. \quad (3.4.2)$$

In dem Quotienten $\frac{n!}{k!(n-k)!}$ werden die letzten $n-k$ Faktoren im Zähler durch $n-k$ Faktoren im Nenner gekürzt, so dass noch k Faktoren im Zähler erhalten bleiben. Also ist $\frac{n!}{k!(n-k)!} < n^k$ und schließlich

$$K(n) = \mathcal{O}(kn^k) = \mathcal{O}(n^k). \quad (3.4.3)$$

Für jeden Agenten gibt es also $\mathcal{O}(n^k)$ mögliche k -beschränkte Gruppen. Die Zeilen 2 - 8 des Algorithmus 3.4.1 werden höchstens $n-1$ mal durchlaufen. Da jede Gruppe aus höchstens k Agenten besteht, ist Zeile 6 in $\mathcal{O}((n-1)kn^k) = \mathcal{O}(n^{k+1})$ ausführbar. In Zeile 9 wird für jede potentielle k -beschränkte Gruppe der Gruppen-Wert mit Algorithmus 3.4.2 berechnet.

In Algorithmus 3.4.2 wird in den Zeilen 1, 2 - 5 und 6 jeweils die Menge der Aufgaben durchlaufen. Das Prüfen, ob noch eine zu bearbeitende Aufgabe existiert, kann in

$\mathcal{O}(kr) + \mathcal{O}(mr)$ durchgeführt werden. Dazu müssen zunächst die Gruppen-Ressourcen B_C mit $B_C = \sum_{a_i \in C} B_i = (b_1^C, \dots, b_r^C)$ in $\mathcal{O}(kr)$ bestimmt werden und anschließend mit jeder Aufgabe in $\mathcal{O}(mr)$ verglichen werden. In den Zeilen 2 - 5 werden für jede bearbeitbare Aufgabe alle r Ressourcen summiert, so dass hierfür ein Rechenaufwand von $\mathcal{O}(mr)$ entsteht. Algorithmus 3.4.2 hat also eine Laufzeit von $\mathcal{O}(m)$. Dadurch hat Algorithmus 3.4.1 eine Laufzeit von $\max \{\mathcal{O}(n^{k+1}), \mathcal{O}(n^k m)\}$, da Algorithmus 3.4.2 von Algorithmus 3.4.1 in Zeile 9 aufgerufen wird.

In jeder Iteration von Algorithmus 3.4.3 wird in den Zeilen 1 - 13 eine Gruppe gebildet und dieser Gruppe eine Aufgabe zugewiesen. Die Zeilen 1 - 13 werden also höchstens m mal durchlaufen. Den größten Gruppen-Wert jeder Gruppe von L in Zeile 2 zu bestimmen und diesen allen Agenten bekannt zu geben, erfordert eine Laufzeit von $\mathcal{O}(n^k)$ und $\mathcal{O}(n)$. Zeile 2 ist also in $\mathcal{O}(mn^k)$ ausführbar. Nachdem jeder Agent $\mathcal{O}(n)$ Gruppen-Werte erhalten hat, kann der größte Gruppen-Wert V unter allen bekannt gegebenen Gruppen-Werten in Zeile 3 durch jeden Agenten in $\mathcal{O}(n)$ bestimmt werden. Für Zeile 3 entsteht also eine Laufzeit von $\mathcal{O}(mn)$. In Zeile 9 wird L für jeden Agenten aus der Gruppe C , die höchstens aus k Agenten besteht, durchlaufen, was in $\mathcal{O}(kn^k)$ realisierbar ist. Zeile 9 ist also in $\mathcal{O}(mn^k)$ ausführbar. In Zeile 10 wird höchstens für jede Gruppe von L der Gruppen-Wert mit Algorithmus 3.4.2 in $\mathcal{O}(n^k m)$ aktualisiert, so dass für Zeile 10 ein Aufwand von $\mathcal{O}(n^k m^2)$ entsteht. Algorithmus 3.4.3 ist somit in $\mathcal{O}(n^k m^2)$ ausführbar.

Insgesamt ergibt sich für die Ausführung von Algorithmus 3.4.1 und Algorithmus 3.4.3 eine Laufzeit von

$$T(m, n) = \max \left\{ \mathcal{O}(n^{k+1}), \mathcal{O}(n^k m^2) \right\}. \quad (3.4.4)$$

Die Abschätzung 3.4.2 gilt für $k \leq \frac{n}{2}$. Tatsächlich sollte für große n jedoch $k \neq k(n)$ und insbesondere $k \ll \frac{n}{2}$ gewählt werden. Es wurde eine Methode implementiert, mit Hilfe derer die Menge aller möglichen k -beschränkten Teilmengen von $\mathcal{A}$ gebildet werden. Die Methode wurde für $k \in \{2, 3, 4\}$ und $n \in [10, 40]$ ausgeführt und die Laufzeit gemessen. In Abschnitt A.1 sind die Messergebnisse festgehalten.

3.4.3. Approximationsgüte

Die Analyse der Approximationsgüte der disjunkten Gruppen ist angelehnt an [SK95] und [SK98].

Für die Analyse der Approximationsgüte ist $c_\nu = \frac{1}{V_\nu}$ der Kosten-Wert einer beliebigen Gruppe C_ν , wobei V_ν der Gruppen-Wert von C_ν ist. Das Gewicht von C_ν ist gegeben durch $w_\nu = \frac{c_\nu}{|C_\nu|}$ (vgl. mit Zeile 6 von Algorithmus 3.4.3). Um die Gesamtkosten zu ermitteln, wird jedem Agenten a_i das Gewicht w_i der Gruppe zugeordnet, der er beigetreten ist, also $w_i = \frac{c_\nu}{|C_\nu|}$ für $a_i \in C_\nu$.

Jedes mal, wenn eine Gruppe C_ν gebildet worden ist, sind ihre Kosten c_ν zu den Gesamtkosten c zu addieren. Dabei steht c für die Gesamtkosten einer beliebigen Gruppen-

Konfiguration $\mathcal{C}$, die sich berechnen durch

$$c = \sum_{a_i \in \mathcal{A}} w_i = \sum_{C_\nu \in \mathcal{C}} \sum_{a_i \in C_\nu} w_i. \quad (3.4.5)$$

Die zweite Gleichheit gilt, da die Reihenfolge der Summanden für das Ergebnis nicht relevant ist und jeweils über alle Agenten einmal summiert wird. Es gilt weiterhin für jede Gruppe C_ν von $\mathcal{C}$:

$$\sum_{a_i \in C_\nu} w_i = \sum_{a_i \in C_\nu} \frac{c_\nu}{|C_\nu|} = c_\nu \sum_{a_i \in C_\nu} \frac{1}{|C_\nu|} \quad (3.4.6)$$

und

$$\sum_{a_i \in C_\nu} \frac{1}{|C_\nu|} \leq \sum_{i=1}^{|C_\nu|} \frac{1}{i}. \quad (3.4.7)$$

Mit 3.4.5, 3.4.6 und 3.4.7 folgt schließlich:

$$c \leq \sum_{C_\nu \in \mathcal{C}} c_\nu \sum_{i=1}^{|C_\nu|} \frac{1}{i}. \quad (3.4.8)$$

Da die Herleitung 3.4.8 für jede beliebige Gruppen-Konfiguration $\mathcal{C}$ gilt, gilt sie insbesondere auch für die optimale Gruppen-Konfiguration $\mathcal{C}^*$:

$$c \leq \sum_{C_\nu \in \mathcal{C}^*} c_\nu \sum_{i=1}^{|C_\nu|} \frac{1}{i} \leq c^* \sum_{i=1}^{\max\{|C_\nu|\}} \frac{1}{i}. \quad (3.4.9)$$

Also ist die Approximationsgüte ρ gegeben durch

$$\rho = \frac{c}{c^*} \leq \sum_{i=1}^{\max\{|C_\nu|\}} \frac{1}{i}, \quad (3.4.10)$$

wobei $\lim_{n \rightarrow \infty} \sum_{i=1}^n \frac{1}{i} = \gamma + \ln n$ und γ die Eulersche Konstante ist ($\gamma = 0,57721 \dots$).

Die Gesamtkosten c einer beliebigen, berechneten Gruppen-Konfiguration $\mathcal{C}$ sind also beschränkt durch $(\gamma + \ln n)c^*$, wobei c^* die Gesamtkosten der optimalen Gruppen-Konfiguration $\mathcal{C}^*$ sind.

3.5. Überlappende Gruppen

Im zweiten Ansatz sollen überlappende Gruppen zugelassen werden. Agenten dürfen also mehreren Gruppen beitreten.

3.5.1. Algorithmus

Der Algorithmus besteht aus zwei wesentlichen Schritten und wird realisiert durch die Algorithmen 3.4.1, 3.4.2, 3.5.1 und 3.5.2. Im ersten Schritt berechnet jeder Agent a_i die potentiellen Gruppen und die Gruppen-Werte mit Algorithmus 3.4.1 und Algorithmus 3.4.2 berechnet. Im zweiten Schritt werden die Gruppen mit Algorithmus 3.5.1 gebildet.

Algorithmus 3.5.1: Bildung der Gruppen

```

1  while  $\mathcal{T} \neq \emptyset$  do
2      Bestimme den größten Gruppen-Wert und gib diesen allen Agenten bekannt
3      Bestimme den größten Gruppen-Wert  $V$  aller Agenten
4      Wähle die zu  $V$  korrespondierende Gruppe  $C$  und Aufgabe  $t$ 
5      if  $a_i \in C$  then
6          Bestimme in  $z$  die Anzahl der neuen Agenten in  $C$ 
7          Gewichte die Gruppe  $C$  mit dem Gewicht  $w = \frac{1}{V_z}$ 
8          Trete der Gruppe  $C$  mit Aufgabe  $t$  bei
9      end
10     Aktualisiere die Ressourcen für jeden Agenten aus  $C$  mit Algorithmus 3.5.2
11     Aktualisiere den Gruppen-Wert  $V$  zu  $\sum_{p=1}^r b_p^t$ 
12     Aktualisiere mit Algorithmus 3.4.2 die Gruppen-Werte der Gruppen, bei denen  $t$ 
        ausgewählt wurde oder irgendein Agent aus  $C$  mitgearbeitet hat
13     Lösche  $t$  aus  $\mathcal{T}$ 
14 end
  
```

Solange noch nicht zugewiesene Aufgaben existieren, werden in den Zeilen 1 - 14 sukzessive die Gruppen gebildet und den entsprechenden Aufgaben zugewiesen. Dabei wird in den Zeilen 2 - 4 die Gruppe C mit dem besten Gruppen-Wert V und der Aufgabe t bestimmt. Jeder Agent a_i gibt dabei mit dem Gruppen-Wert auch die Ressourcen der Agenten aus C in Zeile 2 bekannt. Somit kann jeder Agent die Ressourcen aller Agenten aus C in Zeile 10 aktualisieren, da nicht jeder Agent die Gruppe C auf seiner Liste L noch gespeichert haben muss (vgl. mit Zeile 6 von Algorithmus 3.4.1). Wenn a_i in C enthalten ist, bestimmt a_i in Zeile 6 die Anzahl der neuen Agenten in z . Neue Agenten sind die Agenten, die zum ersten mal einer Gruppe beitreten. Um die Anzahl der z neuen Agenten zu bestimmen, informiert jeder neue Agent $\tilde{a}_i$ aus C alle anderen Agenten aus C , dass er zum ersten mal einer Gruppe beigetreten ist. Agent $\tilde{a}_i$ speichert die Anzahl solcher Meldungen in z und addiert 1 hinzu. In Zeile 7 wird die Gruppe C mit $w = \frac{1}{V_z}$ gewichtet und anschließend der Gruppe C mit Aufgabe t in Zeile 8 beigetreten. Unabhängig davon, ob a_i der Gruppe C beigetreten ist, werden die Ressourcen jedes Agenten der Gruppe C , die Aufgabe t bearbeitet hat, mit Algorithmus 3.5.2 in Zeile 10 aktualisiert. In Zeile 11 wird der Gruppen-Wert V zu $\sum_{p=1}^r b_p^t$ (Summe aller benötigten Ressourcen von t) aktualisiert (vgl. mit Zeile 6 von Algorithmus 3.4.2), da alle nicht benötigten Ressourcen

der Agenten aus C auf die Agenten aus C aufgeteilt wurden. In Zeile 12 müssen wieder die Gruppen-Werte neu berechnet werden, bei denen t ausgewählt wurde. Außerdem muss der Ressourcen-Verlust aller Agenten aus C durch Neuberechnung der Gruppen-Werte der Gruppen berücksichtigt werden, die irgendeinen Agenten aus C enthalten. In Zeile 13 wird die Aufgabe t aus $\mathcal{T}$ gelöscht.

Algorithmus 3.5.2: Aktualisierung der Ressourcen

```

1 foreach  $p = 1, \dots, r$  do
2   foreach  $a_i \in C$  do  $b_p^C \leftarrow b_p^C + b_p^i$ 
3 end
4 foreach  $p = 1, \dots, r$  do
5   foreach  $a_i \in C$  do  $b_p^i \leftarrow \frac{b_p^i}{b_p^C} (b_p^C - b_p^t)$ 
6 end
```

Für jede der $p = 1, \dots, r$ Ressourcen wird von jedem Agenten a_i aus C die p -te Ressource b_p^i in Zeile 5 geändert. Dabei ist b_p^C die Summe der p -ten Ressource aller Agenten aus C , also $b_p^C = \sum_{a_i \in C} b_p^i$, die in den Zeilen 1 - 3 ermittelt wird. Weiterhin ist b_p^t der Bedarf der p -ten Ressource von Aufgabe t . Der Rest $b_p^C - b_p^t$ der p -ten Ressource wird nun so auf alle Agenten aufgeteilt, dass jeder Agent den Teil des Restes $b_p^C - b_p^t$ erhält, mit dem er auch zur Summe der p -ten Ressource aller Agenten beigetragen hat. Die Summe der Anteile des Restes, die jeder Agent erhält, ergibt somit wieder den Rest der p -ten Ressource:

$$\sum_{a_i \in C} \frac{b_p^i}{b_p^C} (b_p^C - b_p^t) = \frac{b_p^C - b_p^t}{b_p^C} \sum_{a_i \in C} b_p^i = b_p^C - b_p^t. \quad (3.5.1)$$

Die nicht benötigten Ressourcen werden also vollständig auf alle Agenten aus C gemäß ihres Beitrages verteilt. Dadurch werden die nicht benötigten Ressourcen jeder Gruppe minimiert. Das folgende Beispiel soll die Arbeitsweise von Algorithmus 3.5.2 verdeutlichen.

Gegeben sei eine Aufgabe t mit den erfordernten Ressourcen $B = (4, 6, 4)$ und 3 Agenten $\{a_1, a_2, a_3\}$, die als Gruppe C die Aufgabe t bearbeitet haben. Dabei hatte a_1 vor der Bearbeitung die Ressourcen $B_1 = (2, 0, 2)$, a_2 die Ressourcen $B_2 = (0, 4, 2)$ und a_3 die Ressourcen $B_3 = (4, 2, 0)$. In Zeile 2 werden zunächst die Gruppen-Ressourcen B_C ermittelt mit $B_C = (6, 6, 4)$. Der Rest der ersten Ressource ist gegeben durch $6 - 4 = 2$. Jeder Agent erhält nun den Teil von 2, mit dem er zur Gesamt-Summe der ersten Ressource beigetragen hat. Also erhält a_1 von 2 genau $\frac{2}{6}$, a_2 von 2 genau $\frac{0}{6}$ und a_3 von 2 genau $\frac{4}{6}$. Die Summe der Ressourcen, die jeder Agent erhält, ergibt wieder den Rest der ersten Ressource, also $\frac{2}{6} \cdot 2 + \frac{0}{6} \cdot 2 + \frac{4}{6} \cdot 2 = 2$. Nachdem die erste Ressource auf die Agenten in Zeile 5 aufgeteilt worden ist, hat a_1 die Ressourcen $B_1 = (\frac{2}{3}, \cdot, \cdot)$, a_2 die Ressourcen $B_2 = (0, \cdot, \cdot)$ und a_3 die Ressourcen $B_3 = (\frac{4}{3}, \cdot, \cdot)$. Wenn auch der Rest der

zweiten und dritten Ressource auf die Agenten aufgeteilt worden ist, hat a_1 die Ressourcen $B_1 = (\frac{2}{3}, 0, 0)$, a_2 die Ressourcen $B_2 = (0, 0, 0)$ und a_3 die Ressourcen $B_3 = (\frac{4}{3}, 0, 0)$.

3.5.2. Laufzeitanalyse

Sei $T(m, n)$ die erforderte Laufzeit bei Ausführung der Algorithmen 3.4.1 und 3.5.1.

Da Algorithmus 3.4.1 in Zeile 9 Algorithmus 3.4.2 aufruft, hat Algorithmus 3.4.1, wie bereits in Abschnitt 3.4.2 ermittelt, eine Laufzeit von $\max \{ \mathcal{O}(n^{k+1}), \mathcal{O}(n^k m) \}$.

Algorithmus 3.5.1 führt die Zeilen 1 - 14 höchstens m mal aus, da nicht mehr als m Aufgaben bearbeitet werden können. In Zeile 6 wird die Anzahl der z neuen Agenten in C nach höchstens $\mathcal{O}(k)$ Mitteilungen der neuen Agenten bestimmt. Zeile 6 ist also in $\mathcal{O}(m)$ ausführbar. Das Aktualisieren der Ressourcen aller Agenten a_i aus C wird mit Algorithmus 3.5.2 in Zeile 10 in $\mathcal{O}(rk)$ realisiert, wodurch Zeile 10 ein Aufwand von $\mathcal{O}(m)$ hat. In Zeile 12 wird mit Algorithmus 3.4.2 höchstens für jede Gruppe von L in $\mathcal{O}(n^k m)$ der Gruppen-Wert neu berechnet. Zeile 12 ist also in $\mathcal{O}(n^k m^2)$ ausführbar. Daraus resultiert für Algorithmus 3.5.1 eine Laufzeit von $\mathcal{O}(n^k m^2)$.

Insgesamt ergibt sich für die Ausführung der Algorithmen 3.4.1 und 3.5.1 eine Laufzeit von

$$T(m, n) = \max \{ \mathcal{O}(n^{k+1}), \mathcal{O}(n^k m^2) \}. \quad (3.5.2)$$

3.5.3. Approximationsgüte

Die Analyse der Approximationsgüte für den Ansatz der überlappenden Gruppen ist angelehnt an [SK95] und [SK98].

Für die Analyse der Approximationsgüte ist $c_\nu = \frac{1}{V_\nu}$ der Kosten-Wert einer beliebigen Gruppe C_ν , wobei V_ν der Gruppen-Wert von C_ν ist. Das Gewicht von C_ν ist gegeben durch $w_\nu = \frac{c_\nu}{z_\nu}$ (vgl. mit Zeile 7 von Algorithmus 3.5.1), wobei z_ν die Anzahl der neuen Agenten $\tilde{a}_i$ von C_ν ist. Neue Agenten $\tilde{a}_i$ der Gruppe C_ν , sind die Agenten, die zum ersten mal einer Gruppe beigetreten sind. Um die Gesamtkosten zu ermitteln, wird jedem Agenten a_i das Gewicht w_i der Gruppe zugeordnet, der er zum ersten mal beigetreten ist, also $w_i = \frac{c_\nu}{z_\nu}$ für $a_i \in C_\nu$.

Jedes mal, wenn eine Gruppe C_ν gebildet worden ist, sind ihre Kosten c_ν zu den Gesamtkosten c zu addieren. Dabei steht c für die Gesamtkosten einer beliebigen Gruppen-Konfiguration $\mathcal{C}$, die sich berechnen durch

$$c = \sum_{a_i \in \mathcal{A}} w_i = \sum_{C_\nu \in \mathcal{C}} \sum_{\tilde{a}_i \in C_\nu} \frac{c_\nu}{z_\nu} = \sum_{C_\nu \in \mathcal{C}} c_\nu \sum_{\tilde{a}_i \in C_\nu} \frac{1}{z_\nu} = \sum_{C_\nu \in \mathcal{C}} c_\nu \sum_{a_i \in C_\nu} \frac{1}{|C_\nu|}. \quad (3.5.3)$$

Damit die Agenten mit ihrem Gewicht nicht mehrfach zu den Gesamtkosten beitragen, wird für jede Gruppe C_ν das Gewicht all der z_ν Agenten summiert, die zum ersten mal einer Gruppe beigetreten sind. Die letzte Gleichheit gilt, da jeweils mit der zweiten

Summe erst z_ν -mal über $\frac{1}{z_\nu}$ summiert wird und dann $|C_\nu|$ -mal über $\frac{1}{|C_\nu|}$. Es gilt weiterhin für jede Gruppe C_ν von $\mathcal{C}$:

$$\sum_{a_i \in C_\nu} \frac{1}{|C_\nu|} \leq \sum_{i=1}^{|C_\nu|} \frac{1}{i}. \quad (3.5.4)$$

Mit 3.5.3 und 3.5.4 folgt schließlich:

$$c \leq \sum_{C_\nu \in \mathcal{C}} c_\nu \sum_{i=1}^{|C_\nu|} \frac{1}{i}. \quad (3.5.5)$$

Da die Herleitung 3.5.5 für jede beliebige Gruppen-Konfiguration $\mathcal{C}$ gilt, gilt sie insbesondere auch für die optimale Gruppen-Konfiguration $\mathcal{C}^*$:

$$c \leq \sum_{C_\nu \in \mathcal{C}^*} c_\nu \sum_{i=1}^{|C_\nu|} \frac{1}{i} \leq c^* \sum_{i=1}^{\max\{|C_\nu|\}} \frac{1}{i}. \quad (3.5.6)$$

Also ist die Approximationsgüte ρ gegeben durch

$$\rho = \frac{c}{c^*} \leq \sum_{i=1}^{\max\{|C_\nu|\}} \frac{1}{i}. \quad (3.5.7)$$

3.6. Gegenüberstellung

Theoretisch unterscheiden sich die Ansätze der disjunkten und überlappenden Gruppen in ihrer Laufzeit und Approximationsgüte nicht. Es ist jedoch zu erwarten, dass die Laufzeit der Algorithmen beim Ansatz der disjunkten Gruppen kleiner ist als beim Ansatz der überlappenden Gruppen, da die Agenten die Berechnung zur Bildung der Gruppen beenden können, sobald sie einer Gruppe beigetreten sind. Beim Ansatz der überlappenden Gruppen kann es sein, dass die Agenten mehreren Gruppen beitreten, sofern die entsprechenden Ressourcen der Agenten und Aufgaben es zulassen. Dadurch, dass die Agenten ihre Ressourcen für die Bearbeitung mehrerer Aufgaben einsetzen können, werden die nicht benötigten Ressourcen verringert. Somit wird der Gruppen-Wert V_ν einer jeden Gruppe C_ν und damit der Gesamterfolg $\sum V_\nu$ erhöht. Es ist also zu erwarten, dass die Ressourcenauslastung beim Ansatz der überlappenden Gruppen mindestens so gut ist wie beim Ansatz der disjunkten Gruppen.

4. Algorithmus von de Weerdt et al.

In diesem Kapitel wird zunächst die Modellierung aus Abschnitt 2.3 ergänzt, so dass das Ziel des Algorithmus von de Weerdt et al. definiert werden kann. Wie schwer das Ziel zu erreichen ist, wird kurz erläutert und anschließend der Algorithmus vorgestellt.

4.1. Modellierung

Die Modellierung aus Abschnitt 2.3 wird so ergänzt, dass das Multiagentensystem, in dem sich die Agenten $\mathcal{A}$ und Aufgaben $\mathcal{T}$ befinden, als ein nicht erfassbares System aufgefasst wird. Es wird also angenommen, dass jeder Agent nur einen Teil von allen Agenten und Aufgaben kennt.

Um diese Situation zu modellieren, wird jeder Agent durch einen Knoten in einem ungerichteten Graphen $G = (\mathcal{A}, E)$ repräsentiert. Dabei ist $\mathcal{A}$ die Menge der Knoten und E die Menge der Kanten von G . Genau dann, wenn eine Kante zwischen zwei Knoten existiert, können die repräsentierten Agenten miteinander kommunizieren. Ein Agent $a_i \in \mathcal{A}$ kann also nur mit seinen Nachbarn $\mathcal{N}(a_i)$ kommunizieren. $\mathcal{N}(a_i)$ enthält alle Agenten, die durch eine Kante mit a_i verbunden sind, also $\mathcal{N}(a_i) = \{a_l \in \mathcal{A} : \{a_i, a_l\} \in E\}$. Abbildung 4.1 zeigt beispielhaft einen Graphen $G = (\mathcal{A}, E)$ mit $\mathcal{A} = \{a_1, a_2, \dots, a_7\}$, in dem die Nachbarn von a_2 gegeben sind durch $\mathcal{N}(a_2) = \{a_1, a_3, a_5, a_6\}$.

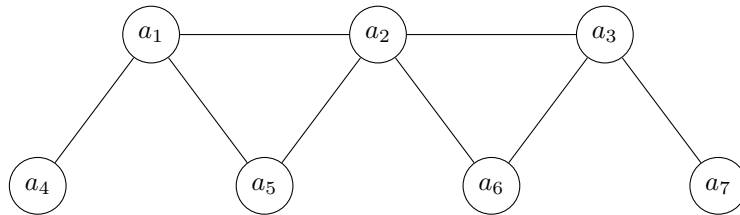


Abbildung 4.1.: Beispiel für einen Graphen $G = (\mathcal{A}, E)$ mit $\mathcal{A} = \{a_1, a_2, \dots, a_7\}$

Ein Agent $a_i \in \mathcal{A}$ hat ganzzahlige, nicht-negative Ressourcen $B_i = (b_1^i, \dots, b_r^i)$ mit $b_p^i \in \mathbb{N}$ für $p = 1, \dots, r$. Die Bearbeitung einer Aufgabe $t_j \in \mathcal{T}$ erfordert ganzzahlige, nicht-negative Ressourcen $B_j = (b_1^j, \dots, b_r^j)$ mit $b_p^j \in \mathbb{N}$ und $\sum_{p=1}^r b_p^j > 0$. Weiterhin wird die Menge der Aufgaben zufällig auf alle Agenten verteilt. Jede Aufgabe $t_j \in \mathcal{T}$ wird durch ein Tripel $t_j = (B_j, u_j, g(t_j))$ dargestellt. Dabei sind B_j die Ressourcen von t_j . Der **Nutzen-Wert** $u_j \in \mathbb{N}$ gibt an, wie groß der Nutzen der Bearbeitung von der Aufgabe t_j ist. Die **Abbildung** g mit $g : \mathcal{T} \rightarrow \mathcal{A}$ gibt an, zu welchem Agenten $g(t_j)$ die

Aufgabe t_j zugewiesen worden ist. Die **Effizienz** e_j einer Aufgabe t_j ist gegeben durch $e_j = \frac{u_j}{\sum b_p^j}$. Je größer also der Nutzen u_j der Aufgabe t_j ist, desto größer ist auch die Effizienz e_j . Je mehr Ressourcen die Bearbeitung der Aufgabe t_j jedoch erfordert, desto kleiner ist die Effizienz e_j . Effiziente Aufgaben sind also die Aufgaben, die einen relativ hohen Nutzen haben und deren Bearbeitungen relativ wenig Ressourcen erfordern.

Die Menge der Aufgaben, die einem Agenten a_i zugewiesen wurde, ist gegeben durch $\mathcal{T}_i = \{t_j \in \mathcal{T} : g(t_j) = a_i\}$. Dabei wird a_i als **Verwalter** der Aufgaben $\mathcal{T}_i$ bezeichnet. Ein Agent a_i kann dabei keine, eine oder mehrere Aufgaben erhalten haben, also $|\mathcal{T}_i| \in \{0, \dots, m\}$. Die Aufgabe $t_{ji} \in \mathcal{T}_i$ mit der größten Effizienz e_{ji} , also $e_{ji} = \max_{t_j \in \mathcal{T}_i} \{e_j\}$, ist die **effizienteste Aufgabe** des Verwalters a_i .

Da die Bearbeitung einer Aufgabe die Ressourcen eines Verwalters übersteigen kann, hat jeder Verwalter die Möglichkeit, sich **ausschließlich durch seine Nachbarn** helfen zu lassen. So hat zum Beispiel Agent a_2 in Abbildung 4.1 die Möglichkeit, sich nur durch die Agenten a_1 , a_3 , a_5 und a_6 helfen zu lassen. Die Menge der Agenten, die einem Verwalter a_i bei der Bearbeitung einer Aufgabe helfen, wird mit $\mathcal{H}(a_i)$ bezeichnet. Dabei sind die Helfer eine Teilmenge der Nachbarn des Verwalters a_i , also $\mathcal{H}(a_i) \subseteq \mathcal{N}(a_i)$. Jeder Agent kann dabei sowohl die Rolle eines Verwalters als auch die Rolle eines Helfers einnehmen. Die **Gesamt-Ressourcen** von a_i und $\mathcal{H}(a_i)$ sind gegeben durch $B_i + (b_1^{\mathcal{H}(a_i)}, \dots, b_r^{\mathcal{H}(a_i)})$ mit $\sum_{a_l \in \mathcal{H}(a_i)} B_l = (b_1^{\mathcal{H}(a_i)}, \dots, b_r^{\mathcal{H}(a_i)})$ (Summe von Vektoren). Eine Aufgabe $t_j \in \mathcal{T}_i$ ist durch a_i und $\mathcal{H}(a_i)$ genau dann **bearbeitbar**, wenn $b_p^j \leq b_p^i + b_p^{\mathcal{H}(a_i)}$ für $p = 1, \dots, r$.

Um den Gesamterfolg des Multiagentensystems zu definieren, sei $\mathcal{T}^* \subseteq \mathcal{T}$ die Teilmenge der Aufgaben, die durch alle Agenten bearbeitet werden. Der **Gesamterfolg** bzw. der **Gesamt-Nutzen** des Multiagentensystems ist dann gegeben durch $\sum_{t_j \in \mathcal{T}^*} u_j$. Je größer der Gesamterfolg ist, desto nützlicher sind die Aufgaben, die die Agenten bearbeiten.

4.2. Ziel

Mit der eingeführten Modellierung lässt sich das Ziel, für das die Agenten bestimmt sind, nun formulieren. Das Ziel besteht darin, eine Teilmenge $\mathcal{T}^* \subseteq \mathcal{T}$ zu finden, so dass der Gesamt-Nutzen $\sum_{t_j \in \mathcal{T}^*} u_j$ maximal wird.

4.3. Komplexität

Shehory und Kraus nehmen an, dass jeder Agent mit jedem anderen Agenten zusammenarbeiten kann. Ihr Algorithmus hat in polynomieller Laufzeit eine logarithmische Approximationsgüte in der Anzahl der Agenten bzgl. der Ressourcenauslastung erzielt. Service und Adams weisen in [SA10] jeder Aufgabe zusätzlich einen Nutzen-Wert zu und zeigen, dass es keinen Algorithmus gibt, der in polynomieller Laufzeit den Gesamt-Nutzen besser als linear in der Anzahl der Aufgaben approximieren kann. de Weerd et

al. fordern, dass ein Agent nur mit seinen Nachbarn zusammenarbeiten kann und dass der Gesamt-Nutzen maximal werden soll. Sie zeigen in [dZK07], dass das Finden einer Teilmenge $\mathcal{T}^* \subseteq \mathcal{T}$ mit maximalem Gesamt-Nutzen NP -vollständig und auch nicht durch einen Algorithmus in polynomieller Laufzeit approximierbar ist in Δ^ε für ein beliebiges $\varepsilon > 0$ (wobei jeder Agent höchstens Δ Nachbarn hat mit $\Delta \geq 3$).

4.4. Algorithmus

Mit dem Algorithmus von de Weerd et al. bestimmen die Agenten die effizientesten Aufgaben und versuchen, diese Aufgaben zusammen zu bearbeiten. Dazu führt jeder Agent a_i Algorithmus 4.4.1 aus, nachdem alle Aufgaben durch g zufällig auf die Agenten verteilt worden sind.

Algorithmus 4.4.1: Bearbeitung der effizientesten Aufgaben

```

1 while  $\mathcal{T}_i \neq \emptyset$  do
2   Bestimme die effizienteste Aufgabe  $t_{ji} \in \mathcal{T}_i$  und gib  $t_{ji}$  allen Nachbarn  $\mathcal{N}(a_i)$  bekannt
3   Bestimme die effizienteste Aufgabe  $t = (B, u, g(t))$  der Nachbarschaft  $\mathcal{N}(a_i) \cup \{a_i\}$ 
4   if  $a_i \neq g(t)$  then Führe Algorithmus 4.4.2 aus
5   else Führe Algorithmus 4.4.3 aus
6 end
```

In Zeile 1 bestimmt jeder Agent a_i zunächst die effizienteste Aufgabe $t_{ji} \in \mathcal{T}_i$ und gibt t_{ji} allen Nachbarn $\mathcal{N}(a_i)$ bekannt. Von allen bekannt gegebenen Aufgaben bestimmt a_i in Zeile 2 die effizienteste Aufgabe $t = (B, u, g(t))$ in seiner Nachbarschaft. Wenn a_i nicht der Verwalter der Aufgabe t ist ($a_i \neq g(t)$), führt a_i Algorithmus 4.4.2 aus, sonst Algorithmus 4.4.3 ($a_i = g(t)$).

Algorithmus 4.4.2: Helfer für t

```

1 Mache dem Verwalter von  $t$  ein Angebot mit entsprechenden Ressourcen für  $t$ 
2 Aktualisiere die Ressourcen entsprechend der Verwendung des Verwalters
```

Wenn a_i nicht der Verwalter der Aufgabe t ist, macht a_i dem Verwalter von t in Zeile 1 ein Angebot mit entsprechenden Ressourcen für t . Dabei beinhaltet das Angebot von a_i alle verfügbaren Ressourcen, die a_i aufbringen kann, um t zu bearbeiten (auch wenn a_i keine verfügbaren Ressourcen mehr hat). Agent a_i wird jedoch nicht mehr Ressourcen anbieten, als für die Bearbeitung von t erforderlich ist.

Der Verwalter $g(t)$ der Aufgabe t prüft für jedes Angebot eines Nachbarn $a \in \mathcal{N}(g(t))$ in Zeile 3, ob t bearbeitbar ist. Wenn t mit den Gesamt-Ressourcen von $g(t)$ und $\mathcal{H}(g(t))$ bearbeitbar ist, wird t bearbeitet. Dazu entscheidet $g(t)$ in Zeile 4, von welchem Helfer welche Ressourcen verwendet werden. In Zeile 5 informiert $g(t)$ jeden Helfer über die Verwendung der Ressourcen. Jeder Helfer aktualisiert daraufhin seine Ressourcen in Zeile

Algorithmus 4.4.3: Verwalter von t

```

1 foreach Angebot von  $a \in \mathcal{N}(g(t))$  do
2    $\mathcal{H}(g(t)) \leftarrow \mathcal{H}(g(t)) \cup \{a\}$ 
3   if  $t$  bearbeitbar then
4     Entscheide, von welchem Helfer welche Ressourcen verwendet werden
5     Informiere die Helfer über die Verwendung ihrer Ressourcen
6     Speichere den Nutzen-Wert  $u$  von  $t$  und lösche  $t$  aus  $\mathcal{T}_{g(t)}$ 
7   end
8 end
9 if  $t$  nicht bearbeitbar und  $|\mathcal{H}(g(t))| = |\mathcal{N}(g(t))|$  then
10  Informiere die Helfer über die Nicht-Verwendung ihrer Ressourcen
11  Lösche  $t$  aus  $\mathcal{T}_{g(t)}$ 
12 end

```

2 von Algorithmus 4.4.2. Anschließend speichert $g(t)$ in Zeile 6 von Algorithmus 4.4.3 den Nutzen-Wert u von t und löscht t aus $\mathcal{T}_{g(t)}$. Wenn t nicht bearbeitet wurde und es Nachbarn $\mathcal{N}(g(t)) \setminus \mathcal{H}(g(t))$ gibt, die $g(t)$ noch kein Angebot gemacht haben, verfügt $g(t)$ solange über die angebotenen Ressourcen der Helfer $\mathcal{H}(g(t))$, bis alle Nachbarn dem Verwalter $g(t)$ ein Angebot gemacht haben. Wenn t dann nicht bearbeitbar ist, werden die Helfer in Zeile 10 über die Nicht-Verwendung ihrer Ressourcen informiert und t wird in Zeile 11 gelöscht. Entsprechend aktualisiert jeder Helfer seine Ressourcen in Zeile 2 von Algorithmus 4.4.2. Die Aufgabe t wird entweder in Zeile 6 oder in Zeile 11 von Algorithmus 4.4.3 gelöscht. Auch wenn t nicht bearbeitet werden konnte, werden alle Nachbarn dem Verwalter $g(t)$ irgendwann ein Angebot machen, da es in ihren Nachbarschaften keine effizienteren Aufgaben mehr zu bearbeiten gibt. Daraufhin wird t entweder bearbeitet und in Zeile 6 gelöscht oder nicht bearbeitet und in Zeile 11 gelöscht.

4.5. Laufzeitanalyse

Sei $T(m, n)$ die erforderte Laufzeit bei Ausführung des Algorithmus 4.4.1.

Die Zeilen 1 - 6 von Algorithmus 4.4.1 werden höchstens m mal durchlaufen, wenn in jeder Iteration nur eine Aufgabe bearbeitet wird. Für das Wählen der effizientesten Aufgabe in Zeile 2 wird zunächst die Effizienz jeder Aufgabe in $\mathcal{O}(r)$ berechnet. Dann wird aus höchstens m Aufgaben die effizienteste gewählt und anschließend allen Nachbarn $\mathcal{N}(a_i)$ gesendet, wobei jeder Agent höchstens $n - 1$ Nachbarn hat. Zeile 2 ist somit in $\max \{\mathcal{O}(m^2), \mathcal{O}(mn)\}$ ausführbar. Zeile 3 ist in $\mathcal{O}(mn)$ ausführbar, da jeder Agent höchstens $n - 1$ Nachbarn hat.

Der Verwalter $g(t)$ wird höchstens von jedem Nachbarn ein Angebot erhalten, so dass die Zeilen 1 - 8 von Algorithmus 4.4.3 höchstens $n - 1$ mal durchlaufen werden. Um in Zeile 3 zu prüfen, ob t bearbeitbar ist, werden die Ressourcen des aktuellen Angebotes zu

den Ressourcen der bereits erhaltenen Angebote in $\mathcal{O}(r)$ addiert. Das Prüfen, ob Aufgabe t bearbeitbar ist, kann in $\mathcal{O}(r)$ realisiert werden, so dass für Zeile 3 ein Aufwand von $\mathcal{O}(mn)$ entsteht. Der Verwalter $g(t)$ entscheidet in Zeile 4 in $\mathcal{O}(rn)$, von welchem Helfer welche Ressourcen verwendet werden. Zeile 4 ist also in $\mathcal{O}(mn^2)$ ausführbar. Alle Helfer können in $\mathcal{O}(n)$ bzgl. der Verwendung der Ressourcen in Zeile 5 informiert werden. Zeile 5 ist also in $\mathcal{O}(mn^2)$ ausführbar. Die Zeile 6 kann in $\mathcal{O}(m^2n)$ realisiert werden, da t erst gesucht werden muss und anschließend gelöscht werden kann. Das Prüfen, ob t nicht bearbeitbar ist und ob $|\mathcal{H}(g(t))| = |\mathcal{N}(g(t))|$, kann in $\mathcal{O}(r)$ und $\mathcal{O}(n)$ durchgeführt werden, so dass für Zeile 9 ein Aufwand von $\mathcal{O}(mn)$ entsteht. Zeile 10 kann in $\mathcal{O}(mn)$ durchgeführt werden, da $g(t)$ höchstens $n - 1$ Nachbarn hat.

Jeder Helfer kann $g(t)$ in $\mathcal{O}(r)$ ein entsprechendes Angebot machen. Zeile 1 von Algorithmus 4.4.2 ist also in $\mathcal{O}(m)$ ausführbar. Das Aktualisieren der Ressourcen kann in $\mathcal{O}(r)$ realisiert werden. Jeder Helfer muss dabei die Laufzeit von Algorithmus 4.4.3 für höchstens $m - 1$ Aufgaben abwarten, bis er seine Ressourcen aktualisieren kann. Denn nach höchstens $m - 1$ Aufgaben wird die aktuelle Aufgabe erneut auf Bearbeitung überprüft. Algorithmus 4.4.3 hat für $m - 1$ Aufgaben eine Laufzeit von $\max \{ \mathcal{O}(mn^2), \mathcal{O}(m^2n) \}$, so dass Zeile 2 von Algorithmus 4.4.2 in $\max \{ \mathcal{O}(m^2n^2), \mathcal{O}(m^3n) \}$ ausführbar ist.

Insgesamt ergibt sich für die Ausführung von Algorithmus 4.4.1 eine Laufzeit von

$$T(m, n) = \max \{ \mathcal{O}(m^2n^2), \mathcal{O}(m^3n) \}. \quad (4.5.1)$$

4.6. Qualität der Lösung

de Weerd et al. geben für ihren Algorithmus keine Güte an, mit der sich eine Aussage über die Qualität der berechneten Lösung ihres Algorithmus (in [dZK07] greedy distributed allocation protocol, GDAP) machen lässt. Um dennoch eine Aussage über die Qualität der Lösung des Algorithmus zu machen, haben de Weerd et al. den Gesamt-Nutzen, den ihr Algorithmus erzielt, verglichen mit dem Gesamt-Nutzen, der durch zwei verschiedene Algorithmen erzielt wird.

Der erste Algorithmus (in [dZK07] optimal social task allocation, OPT) ist ein Algorithmus, der in exponentieller Laufzeit für kleine Eingaben n, m eine optimale Lösung bzw. den maximalen Gesamt-Nutzen erzielt. Der zweite Algorithmus (in [dZK07] an upper bound for social task allocation, UB) ist ein Algorithmus, der in polynomieller Laufzeit für größere Eingaben n, m eine obere Schranke des Gesamt-Nutzens erzielt. de Weerd et al. haben die drei Algorithmen GDAP, OPT und UB implementiert, simuliert und die Ergebnisse der Simulation in [dZK07] dargestellt.

5. Implementierung und Simulation

Die Algorithmen von Shehory und Kraus sowie von de Weerd et al. wurden implementiert und simuliert. Die Implementierung und die Ergebnisse der Simulation werden in diesem Kapitel vorgestellt.

5.1. Implementierung

Die Algorithmen von Shehory und Kraus und von de Weerd et al. wurden in der Programmiersprache Java implementiert. Für den Algorithmus von Shehory und Kraus wurden dabei die Klassen `Agent`, `Task` und `Group` erstellt. Die Klasse `Agent` verfügt u.a. über eine Methode `setGroups(Set agents, int k)`, mit Hilfe derer die Agenten alle möglichen k -beschränkten Teilmengen der Agenten bilden. Es wurden zwei Klassen `DisjointGroups` und `OverlappingGroups` implementiert, die die abstrakte Klasse `ShehoryKraus` erweitern und jeweils den Algorithmus für die disjunkten und überlappenden Gruppen durch die Methode `algorithm()` implementieren. Die Klasse `ShehoryKrausSimulator` bildet jeweils Instanzen der Klassen `DisjointGroups` und `OverlappingGroups` und simuliert die Algorithmen durch den Aufruf der jeweiligen Methode `algorithm()`.

Für den Algorithmus von de Weerd et al. wurde die Klasse `Agent` und die Klasse `Task` erstellt. Dabei wurde mit der Klasse `Agent` u.a. die Methode `getMostEfficientTaskOfNeighborhood()` implementiert, mit Hilfe derer die Agenten die effizienteste Aufgabe in ihrer Nachbarschaft bestimmen. Weiterhin wurde eine Klasse `deWeertZhangKlos` erstellt, die zum Beispiel mit der Methode `algorithm()` den Algorithmus von de Weerd et al. implementiert. Die Klasse `deWeertZhangKlosSimulator` erstellt eine Instanz der Klasse `deWeertZhangKlos`, für welche die Methode `algorithm()` aufgerufen wird, so dass dadurch der Algorithmus von Weerd et al. simuliert wird.

5.2. Simulation

Nachfolgend werden die Ergebnisse der Simulation der implementierten Algorithmen vorgestellt und analysiert.

5.2.1. Algorithmus von Shehory und Kraus

Der Algorithmus von Shehory und Kraus wurde für $n = 10$ Agenten, $m = 10$ Aufgaben, $r = 5$ Ressourcen und $k \in \{1, 2, \dots, n\}$ simuliert. Die Agenten haben jede der 5

Ressourcen zufällig aus einem ganzzahligen Intervall $[0, 3]$ erhalten. Die Aufgaben haben jede der 5 Ressourcen zufällig aus einem ganzzahligen Intervall $[0, 6]$ erhalten. Dabei haben die Agenten und Aufgaben die Ressourcen einmal zufällig erhalten und für jedes $k \in \{1, 2, \dots, n\}$ identisch beibehalten. Die Abbildung 5.1 zeigt die Ergebnisse der Simulation für die disjunkten und überlappenden Gruppen.

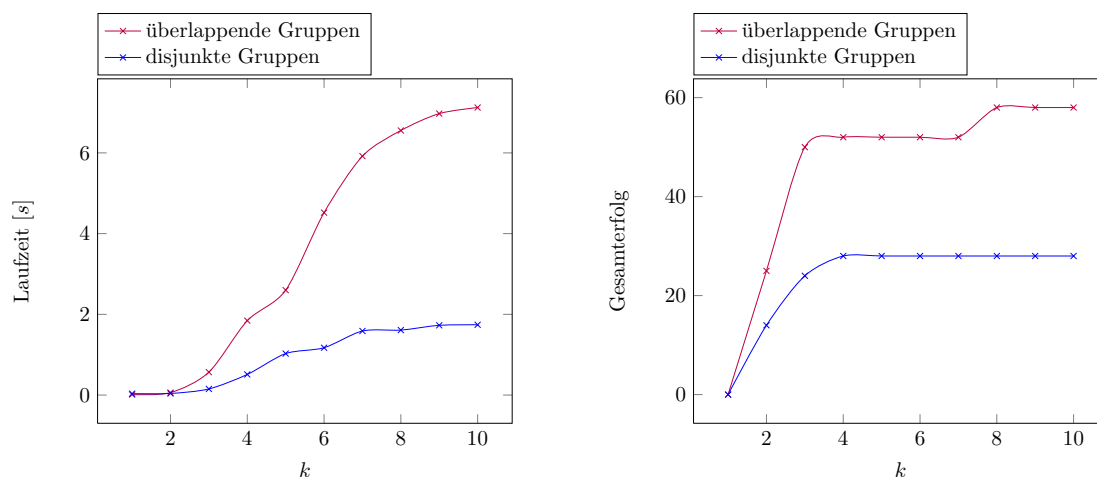


Abbildung 5.1.: Laufzeit und Gesamterfolg in Abhängigkeit von k

In Abbildung 5.1 ist zu erkennen, dass die Laufzeit für jedes $k \in \{1, 2, \dots, n\}$ beim Ansatz der disjunkten Gruppen gleich oder kleiner ist, wie beim Ansatz der überlappenden Gruppen. Dies ist darin begründet, dass die Agenten beim Ansatz der disjunkten Gruppen ihre Berechnung beenden können, so bald sie einer Gruppe beigetreten sind. Beim Ansatz der überlappenden Gruppen können Agenten mehreren Gruppen beitreten. Außerdem müssen die Ressourcen der Agenten, nachdem sie einer Gruppe beigetreten sind, aktualisiert werden. Die nicht benötigten Ressourcen werden auf alle Agenten der jeweiligen Gruppen aufgeteilt, wodurch der Gruppen-Wert V_ν jeder Gruppe C_ν und damit der Gesamterfolg $\sum V_\nu$ erhöht wird.

Es lässt sich außerdem beobachten, dass sich der Gesamterfolg für $k \geq 4$ bei beiden Ansätzen fast nicht ändert. Das liegt daran, dass die Gruppen-Ressourcen durch die Aufnahme weiterer Agenten größer werden. Der Gruppen-Wert von kleineren Gruppen, die bereits Aufgaben bearbeiten können, wird durch die Aufnahme weiterer Agenten nur schlechter, da noch mehr Ressourcen nicht benötigt werden.

Bei den überlappenden Gruppen ist der Gesamterfolg für $k = 8$ erhöht worden. Die Ressourcen der Agenten werden entsprechend der Verwendung aktualisiert und kleiner. Somit kann es sein, dass große Gruppen nur noch wenig Gruppen-Ressourcen haben und durch die Aufnahme weiterer Agenten nun wieder als Gruppe eine Aufgabe bearbeiten können.

Der Gesamterfolg der disjunkten Gruppen für $k = 2$ beträgt 14 (vgl. mit Abbil-

dung 5.1). Die Gesamtkosten c (vgl. mit Abschnitt 3.4.3) berechnen sich dann zu $c = \frac{1}{14} = 0.071$ (gerundet auf die dritte Nachkommastelle). Die minimalen Kosten sind mit $k = 10$ gegebenen durch $c^* = \frac{1}{28} = 0.036$. Für $n = 10$ und $\gamma = 0.577$ sind die Gesamtkosten c für $k = 2$ beschränkt durch $(\gamma + \ln n)c^*$, also $c \leq (\gamma + \ln n)c^*$ bzw. $0.071 \leq 0.103$. Der Gesamterfolg der überlappenden Gruppen für $k = 2$ beträgt 25 (vgl. mit Abbildung 5.1). Die Gesamtkosten c (vgl. mit Abschnitt 3.5.3) berechnen sich dann zu $c = \frac{1}{25} = 0.04$. Die minimalen Kosten sind mit $k = 10$ gegebenen durch $c^* = \frac{1}{58} = 0.017$. Für $n = 10$ und $\gamma = 0.577$ sind die Gesamtkosten c für $k = 2$ beschränkt durch $(\gamma + \ln n)c^*$, also $c \leq (\gamma + \ln n)c^*$ bzw. $0.04 \leq 0.049$.

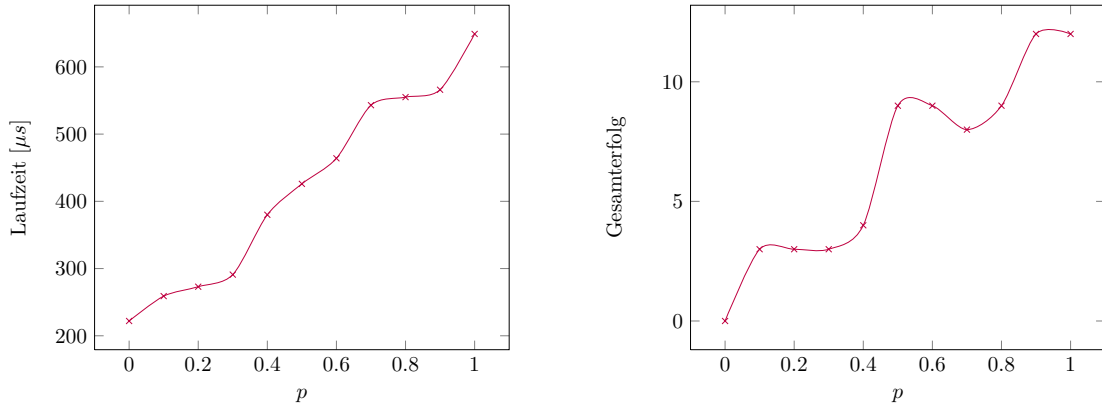
5.2.2. Algorithmus von de Weerd et al.

Um den Algorithmus von de Weerd et al. zu simulieren, wurde ein Zufalls-Graph (angelehnt an [ER59]) erstellt, in dem jeder Agent durch einen Knoten repräsentiert wird. Mit Wahrscheinlichkeit $p \in [0, 1]$ sind dabei je zwei Agenten miteinander verbunden. Der Algorithmus von de Weerd et al. wurde für $n = 10$ Agenten, $m = 10$ Aufgaben, $r = 5$ Ressourcen und $p \in \{0.0, 0.1, \dots, 1.0\}$ simuliert. Die Agenten haben jede der 5 Ressourcen zufällig aus einem ganzzahligen Intervall $[0, 3]$ erhalten. Die Aufgaben haben jede der 5 Ressourcen zufällig aus einem ganzzahligen Intervall $[0, 6]$ und jeweils einen zufälligen Nutzen-Wert aus einem ganzzahligen Intervall $[0, 3]$ erhalten. Dabei haben die Agenten und Aufgaben die Ressourcen bzw. Nutzen-Werte einmal zufällig erhalten und für jedes $p \in \{0.0, 0.1, \dots, 1.0\}$ identisch beibehalten.

Bei der Simulation ist die Laufzeit für die erste Ausführung des Algorithmus (mit den Graphen für $p = 0.0$) die größte Laufzeit gewesen. Um jedoch eine präzise Aussage zur Laufzeit zu machen, wurde die Simulation 1000 mal wiederholt und der Mittelwert für die Laufzeit für jedes $p \in \{0.0, 0.1, \dots, 1.0\}$ gebildet. Dabei wurden die Ressourcen der Agenten und Aufgaben sowie die Nachbarn für jeden Agenten für jede Wiederholung identisch beibehalten, so dass sich der Gesamterfolg für jede Wiederholung nicht geändert hat. Die Abbildung 5.2 zeigt die Ergebnisse der Simulation.

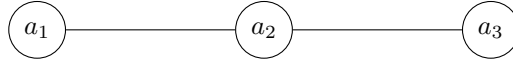
Es fällt in Abbildung 5.2 auf, dass die Laufzeit mit wachsendem p größer wird. Je größer p ist, desto wahrscheinlicher ist es, dass je zwei Agenten miteinander verbunden sind. So ist zum Beispiel für $p = 1.0$ der Graph, in dem sich die Agenten befinden, vollständig. Jeder Agent muss für jeden Nachbarn prüfen, welches die effizienteste Aufgabe aller Nachbarn ist. Für $p = 1.0$ hat jeder Agent die Aufgaben aller anderen Agenten zu überprüfen. Somit ist für $p = 1.0$ die Laufzeit im Mittel am größten.

Es lässt sich auch beobachten, dass der Gesamterfolg $\sum_{t_j \in \mathcal{T}^*} u_j$ mit wachsendem p tendenziell größer wird. Je größer p wird, desto wahrscheinlicher ist es, dass die effizientesten Aufgaben bearbeitet werden können und somit der Gesamterfolg steigt. Da jeder Agent durch alle anderen Agenten für $p = 1.0$ unterstützt werden kann, können die effizientesten Aufgaben bearbeitet werden. Jedoch muss es nicht so sein, dass für einen

Abbildung 5.2.: Laufzeit und Gesamterfolg in Abhängigkeit von p

vollständigen Graphen der Algorithmus von de Weerd et al. den maximalen Gesamterfolg erzielt. Das nachfolgende Beispiel soll das verdeutlichen.

Gegeben seien 3 Agenten $\{a_1, a_2, a_3\}$ mit den Ressourcen $B_1 = (1, 1, 1)$, $B_2 = (2, 1, 1)$ und $B_3 = (2, 1, 1)$. Weiterhin seien 3 Aufgaben $\{t_1, t_2, t_3\}$ mit $t_1 = ((1, 3, 0), 3, a_1)$, $t_2 = ((2, 1, 1), 2, a_2)$ und $t_3 = ((2, 1, 1), 2, a_3)$ gegeben. Damit ergibt sich für die Aufgaben jeweils eine Effizienz von $e_1 = \frac{3}{4}$, $e_2 = \frac{1}{2}$ und $e_3 = \frac{1}{2}$. Wenn der Graph, in dem sich die Agenten befinden, nun vollständig ist, erzielen die Agenten einen Gesamterfolg von $u_1 = 3$. Da zuerst t_1 bearbeitet und dabei von allen Agenten die zweite Ressource verbraucht wird, ist keine andere Aufgabe mehr bearbeitbar. Wenn der Graph G , in dem sich die Agenten befinden, unvollständig ist mit $G = (\{a_1, a_2, a_3\}, \{\{a_1, a_2\}, \{a_2, a_3\}\})$, dann

Abbildung 5.3.: Veranschaulichung des Graphen G

werden a_1 und a_2 versuchen, zuerst t_1 zu bearbeiten. Da a_1 und a_2 die Aufgabe t_1 nicht bearbeiten können, wird t_1 gelöscht und anschließend können durch a_1 , a_2 und a_3 die Aufgaben t_2 und t_3 bearbeitet werden. Somit entsteht ein Gesamterfolg von $u_2 + u_3 = 4$.

Für $p = 0.7$ ist der Gesamterfolg kleiner als der Gesamterfolg bei $p = 0.6$ (vgl. mit Abbildung 5.2). Einerseits hat das soeben beschriebene Beispiel gezeigt, dass mit wachsendem p nicht immer auch der Gesamterfolg größer werden muss. Andererseits ist die erwartete Anzahl an Kanten in dem Graphen für $p = 0.7$ zwar größer als in dem Graphen für $p = 0.6$ (vgl. mit Abschnitt A.2), jedoch ist nicht auszuschließen, dass in dem Graphen für $p = 0.6$ mehr Kanten existieren als in dem Graphen für $p = 0.7$.

6. Zusammenfassung

In dieser Arbeit wurden die Modelle von Shehory und Kraus und von de Weerd et al. zur Gruppenbildung für die Aufgabenverteilung in Multiagentensystemen vorgestellt. Beide Modelle gehen davon aus, dass die Agenten Ressourcen haben, um die Aufgaben, deren Bearbeitung Ressourcen erfordert, zu bearbeiten. Wenn ein Agent auf Grund seiner beschränkten Ressourcen eine Aufgabe nicht bearbeiten kann, hat er die Möglichkeit, diese Aufgabe in einer Gruppe von Agenten zu bearbeiten. Mit dem Algorithmus von Shehory und Kraus werden die Gruppen so gebildet, dass die Gruppen-Ressourcen, welche sich aus der Summe der Ressourcen jedes Agenten dieser Gruppe ergeben, so zu einer Aufgabe passen, dass möglichst wenig Ressourcen nicht genutzt werden. Das Ziel des Algorithmus von Shehory und Kraus besteht also darin, die Gruppen so zu bilden, dass die ungenutzten Ressourcen der Gruppen-Ressourcen bzgl. der zu bearbeitenden Aufgabe minimiert werden. de Weerd et al. weisen jeder Aufgabe ein Nutzen-Wert zu. Das Ziel des Algorithmus von de Weerd et al. besteht darin, den Gesamt-Nutzen, der sich aus der Summe der Nutzen-Werte der bearbeiteten Aufgaben ergibt, zu maximieren. Die Algorithmen von Shehory und Kraus sowie von de Weerd et al. wurden in dieser Arbeit beschrieben, ihre Laufzeit analysiert und implementiert.

6.1. Fazit

Mit dem Algorithmus von Shehory und Kraus bilden die Agenten alle möglichen k -beschränkten Teilmengen von der Menge der Agenten. Die Laufzeit ihres Algorithmus wurde analysiert zu $T(m, n) = \max \{ \mathcal{O}(n^{k+1}), \mathcal{O}(n^k m^2) \}$, wobei n für die Anzahl der Agenten und m für die Anzahl der Aufgaben steht. In dem Modell von de Weerd et al. dürfen die Agenten zur Bearbeitung einer Aufgabe sich nur durch ihre Nachbarn helfen lassen. Die Laufzeit des Algorithmus von de Weerd et al. wurde analysiert zu $T(m, n) = \max \{ \mathcal{O}(m^2 n^2), \mathcal{O}(m^3 n) \}$. Beide Algorithmen sind in dieser Arbeit für $n = 10$ Agenten und $m = 10$ Aufgaben simuliert und die Laufzeit jeweils gemessen worden. Der Algorithmus von de Weerd et al. hat mit den verwendeten Parametern bei der Simulation die kleinere Laufzeit (vgl. mit Abbildung 5.1 und mit Abbildung 5.2). Für Multiagentensysteme, in denen sich sehr viele Agenten befinden, bietet sich also die Verwendung des Modells von de Weerd et al. an. Einerseits ist die Laufzeit ihres Algorithmus kleiner, andererseits ist es in großen Multiagentensystemen (mit sehr vielen Agenten) eher so, dass nicht jeder Agent alle anderen Agenten kennt und somit auch nicht mit allen an-

deren Agenten kommunizieren und zusammenarbeiten kann. Für Multiagentensysteme, in denen sich wenig Agenten befinden, bietet sich die Verwendung des Modells von Shehory und Kraus an. Sie gehen davon aus, dass jeder Agent mit jedem anderen Agenten kommunizieren und zusammenarbeiten kann. Ihr Algorithmus erzielt eine logarithmische Approximationsgüte in der Anzahl der Agenten bzgl. der Ressourcenauslastung.

6.2. Ausblick

Um die Laufzeit des Algorithmus von de Weerd et al. als Maß für eine erstrebenswerte Laufzeit zu wählen, könnte das Modell von Shehory und Kraus mit dem Modell von den Weerd et al. wie folgt kombiniert werden. Die Agenten in dem Modell von Shehory und Kraus befinden sich nun in einer nicht erfassbaren Umgebung. Jeder Agent darf nur mit den Agenten eine Gruppe bilden, die sich auch in seiner Nachbarschaft befinden. Für dieses Modell könnte der Algorithmus von Shehory und Kraus modifiziert und die Laufzeit analysiert werden.

In Abschnitt 2.1.1 wurde ein Agent unter anderem damit charakterisiert, dass er fähig ist, zu lernen. Angenommen, die Agenten können nur mit ihren Nachbarn zusammenarbeiten. Dann könnte untersucht werden, wie die Agenten lernen können, mit guten Nachbarn zusammen zu arbeiten, um dadurch den Gesamterfolg des Multiagentensystems zu vergrößern. Der Algorithmus dieses Modells wird für einen Graphen, in dem jeder Agent durch einen Knoten repräsentiert wird, ausgeführt. Bei der nächsten Ausführung des Algorithmus halten die Agenten nur an den Nachbarn fest, die sich bei der vorangegangenen Ausführung des Algorithmus für sie bei der Zusammenarbeit als besonders gut erwiesen haben.

A. Anhang

A.1. Messergebnisse

In Zeile 1 von Algorithmus 3.4.1 werden alle möglichen k -beschränkten Teilmengen von $\mathcal{A}$ bestimmt. Dazu wurde eine Methode implementiert, mit Hilfe derer die Menge aller

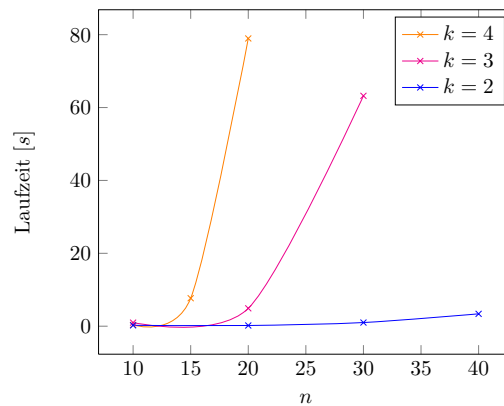


Abbildung A.1.: Laufzeit in Abhängigkeit von n für $k \in \{2, 3, 4\}$

möglichen k -beschränkten Teilmengen von $\mathcal{A}$ gebildet werden. Die Methode wurde für $k \in \{2, 3, 4\}$ und $n \in [10, 40]$ ausgeführt und die Laufzeit in Sekunden gemessen. Das Diagramm in Abbildung A.1 zeigt die Laufzeit in Abhängigkeit von n für $k \in \{2, 3, 4\}$.

A.2. Zufalls-Graph

Um den Algorithmus von de Weerd et al. zu simulieren, wurde ein Zufalls-Graph (angelehnt an [ER59]) erstellt, in dem jeder Agent durch einen Knoten repräsentiert wird. Mit Wahrscheinlichkeit $p \in [0, 1]$ sind dabei je zwei Agenten miteinander verbunden.

Um die erwartete Anzahl an Kanten in G für p zu bestimmen, wird X_i als binäre Zufallsvariable definiert mit

$$X_i = \begin{cases} 1 & \text{falls Kante } i \text{ gewählt} \\ 0 & \text{sonst.} \end{cases} \quad (\text{A.2.1})$$

Der Erwartungswert von X_i berechnet sich mit

$$E(X_i) = \sum_{k=0}^1 k \cdot P(X_i = k) = p. \quad (\text{A.2.2})$$

Insgesamt können für G maximal $\binom{n}{2}$ Kanten gewählt werden. Um die erwartete Anzahl an Kanten in G für p zu bestimmen, wird die Zufallsvariable X definiert mit

$$X = \sum_{i=1}^{\binom{n}{2}} X_i \quad (\text{A.2.3})$$

Die erwartete Anzahl an Kanten in G für p berechnet sich dann mit

$$E(X) = E\left(\sum_{i=1}^{\binom{n}{2}} X_i\right) = \sum_{i=1}^{\binom{n}{2}} E(X_i) = \binom{n}{2} p. \quad (\text{A.2.4})$$

Für $n = 10$ und $p = 0.6$ berechnet sich die erwartete Anzahl an Kanten in G zu $E(X) = 27$. Für $p = 0.7$ berechnet sich die erwartete Anzahl an Kanten in G zu $E(X) = 31.5$. Im Mittel hat G für $p = 0.6$ also 4.5 Kanten weniger als G für $p = 0.7$.

A.3. CD

Der Arbeit liegt eine CD mit folgender Verzeichnisstruktur bei.

- **Ausarbeitung.pdf** - elektronische Version der schriftlichen Ausarbeitung.
- **Literatur** - dieses Verzeichnis enthält alle verwendeten Quellen, die als pdf-Dokument erreicht werden konnten.
- **Quellcode/ShehoryKraus/src** - in diesem Verzeichnis befinden sich die implementierten Java-Klassen des Algorithmus von Shehory und Kraus.
- **Quellcode/ShehoryKraus/doc** - in diesem Verzeichnis befindet sich die Dokumentation zu den implementierten Java-Klassen des Algorithmus von Shehory und Kraus. Die Dokumentation wurde mit Javadoc erstellt und kann durch Aufruf der **index.html** eingesehen werden.
- **Quellcode/deWeerdZhangKlos/src** - in diesem Verzeichnis befinden sich die implementierten Java-Klassen des Algorithmus von de Weerd et al..
- **Quellcode/deWeerdZhangKlos/doc** - in diesem Verzeichnis befindet sich die Dokumentation zu den implementierten Java-Klassen des Algorithmus von de Weerd et al. Die Dokumentation kann durch Aufruf der **index.html** eingesehen werden.
- **Quellcode/jar** - in diesem Verzeichnis befinden sich die implementierten Algorithmen von Shehory und Kraus sowie von de Weerd et al. als ausführbare jar-Dateien (**ShehoryKraus.jar** und **deWeerdZhangKlos.jar**). Eine Anleitung zur Ausführung der jar-Dateien befindet sich ebenfalls in diesem Verzeichnis (**Anleitung.pdf**).

Literaturverzeichnis

- [dZK07] DE WEERDT, Mathijs ; ZHANG, Yingqian ; KLOS, Tomas: Distributed task allocation in social networks. In: *AAMAS '07: Proceedings of the 6th international joint conference on Autonomous agents and multiagent systems*, ACM, 2007. – ISBN 978-81-904262-7-5, S. 1–8
- [ER59] ERDŐS, P. ; RÉNYI, A.: On random graphs, I. In: *Publ. Math. Debrecen* 6 (1959)
- [Kar72] KARP, Richard M.: Reducibility Among Combinatorial Problems. In: *Complexity of Computer Computations* (1972), S. 85–103
- [Koz92] KOZEN, Dexter C.: *The design and analysis of algorithms*. Springer, 1992
- [RN04] RUSSELL, Stuart ; NORVIG, Peter: *Künstliche Intelligenz: Ein moderner Ansatz*. 2. Pearson Studium, 2004
- [SA10] SERVICE, Travis C. ; ADAMS, Julie A.: Coalition formation for task allocation: theory and algorithms. In: *Autonomous Agents and Multi-Agent Systems* (2010)
- [SK93] SHEHORY, Onn ; KRAUS, Sarit: Coalition formation among autonomous agents: Strategies and complexity. In: *MAAMAW*, Springer, 1993, S. 56–72
- [SK95] SHEHORY, Onn ; KRAUS, Sarit: Task Allocation via Coalition Formation Among Autonomous Agents. In: *Proc. of IJCAI*, 1995, S. 655–661
- [SK96] SHEHORY, Onn ; KRAUS, Sarit: *Formation of Overlapping Coalitions for Precedence-Ordered Task-Execution Among Autonomous Agents*. 1996
- [SK98] SHEHORY, Onn ; KRAUS, Sarit: Methods for task allocation via agent coalition formation. In: *Artificial Intelligence* 101 (1998), Mai, Nr. 1, S. 165–200
- [Syc98] SYCARA, Katia: Multiagent Systems. In: *AI Magazine* 10 (1998), Nr. 1, S. 79–93
- [TA05] TOSIC, Predrag T. ; AGHA, Gul A.: Maximal Clique Based Distributed Coalition Formation for Task Allocation in Large-Scale Multi-agent Systems. In: *Massively Multi-Agent Systems I*, Springer, 2005, S. 104–120

- [Woo98] WOOLDRIDGE, Michael J.: Agent-based computing. In: *Interoperable Communication Networks* (1998), Januar, S. 71–97
- [Woo09] WOOLDRIDGE, Michael J.: *MultiAgent systems : An Introduction*. 2. Chichester : Wiley, 2009